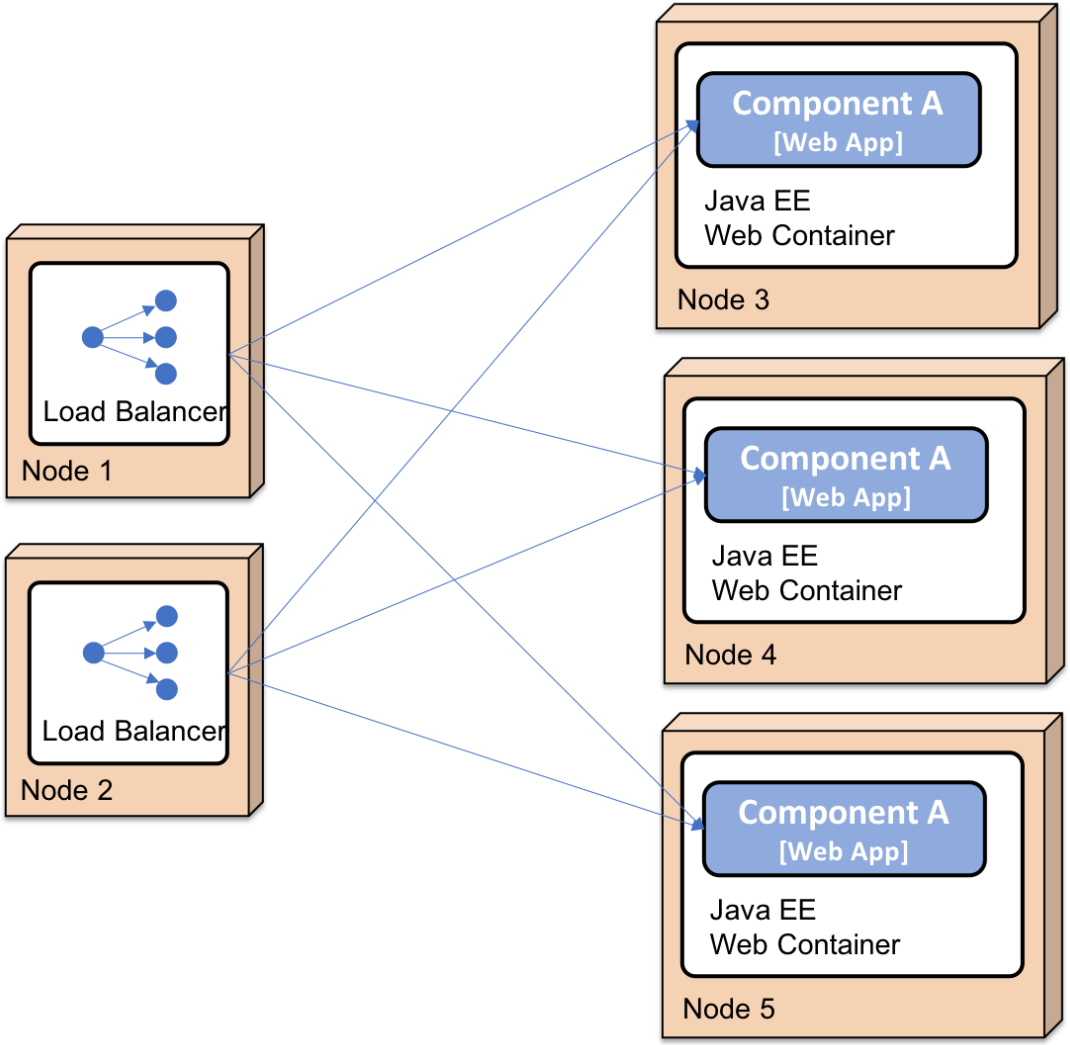
2강. MSA 개념과 설계원칙

한국공학대학교
TECH UNIVERSITY OF KOREA

이영곤

» 마이크로서비스 디자인 패턴

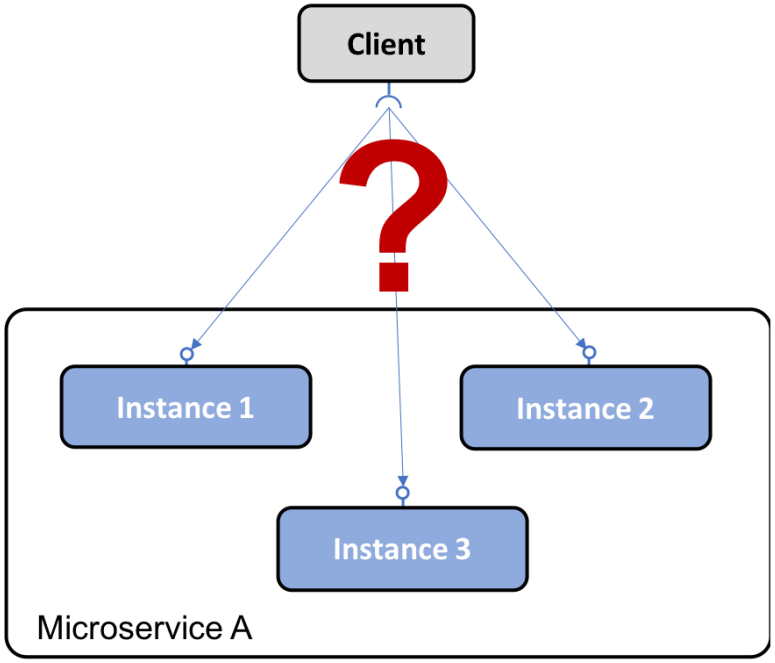
- 서비스 검색
- 에지 서버
- 리액티브 마이크로서비스
- 구성 중앙화
- 로그 분석 중앙화
- 분산 추적
- 서킷 브레이커
- 제어 루프
- 모니터링 및 경고 중앙화



» 서비스 검색

문제점

- 컨테이너에서 실행되는 마이크로서비스 인스턴스는 시작하면서 동적 IP 주소 할당 받음
- 마이크로서비스가 노출하는 HTTP 기반의 REST API를 클라이언트에서 호출하기 어렵게 함



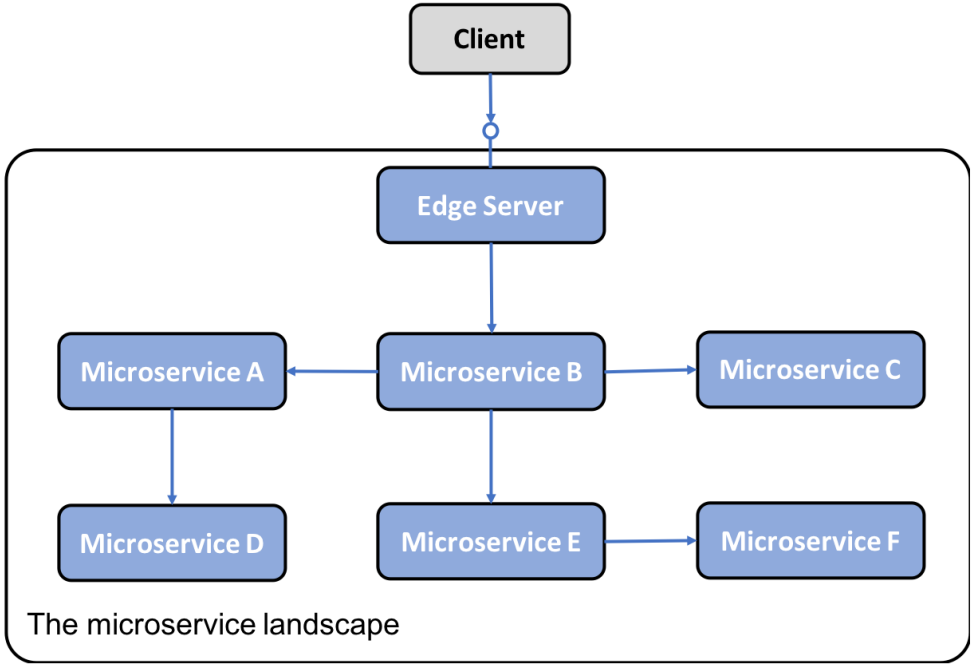
해결책

- 마이크로서비스와 인스턴스를 자동으로 등록 및 해지
- 클라이언트는 마이크로서비스의 논리 엔드포인트에 요청 보낼 수 있어야 함
- 마이크로서비스에 대한 요청은 가용 인스턴스로 로드밸런싱 되어야 함
- 비정상적인 인스턴스 감지가 가능해야 함

» 에지 서버

- 문제점
  - 마이크로서비스중 일부만 외부에 공개하고 그외의 마이크로서비스는 외부에서 접근하지 못하도록 숨겨야 함
  - 악의적인 클라이언트 요청에서 보호해야 함

- 해결책
  - 모든 요청이 통과하는 새 컴포넌트(에지서버)를 추가함
  - 표준 프로토콜과 Oauth, OIDC, JWT 토큰, API 키 등을 활용해 클라이언트 신뢰성 체크



» 리액티브 마이크로서비스

문제점

- HTTP기반의 Rest JSON API와 같은 블로킹 I/O 사용시 운영체제 스레드를 점유하여 요청수 증가시 가용 스레드 부족으로 응답 속도 느려지고 서버 중단 사례 발생

해결책

- 논블로킹 I/O를 사용해 데이터베이스나 다른 마이크로서비스가 처리하길 기다리는 동안 스레드가 할당되지 않게 함
- 비동기식 모델 사용하거나 리액티브 프레임워크를 사용
- 어떤 서비스가 중단되더라도 응답할 수 있는 탄력성을 가지도록 설계해야 함

» 구성 중앙화

- 문제점
  - 실행중인 모든 마이크로서비스 인스턴스의 구성 정보를 한눈에 보는 방안 필요
  - 구성을 업데이트하고 관련된 모든 마이크로서비스 인스턴스가 일괄 업데이트 되게 하려면?

- 해결책
  - 모든 마이크로서비스의 구성 정보를 저장하는 컴포넌트를 추가

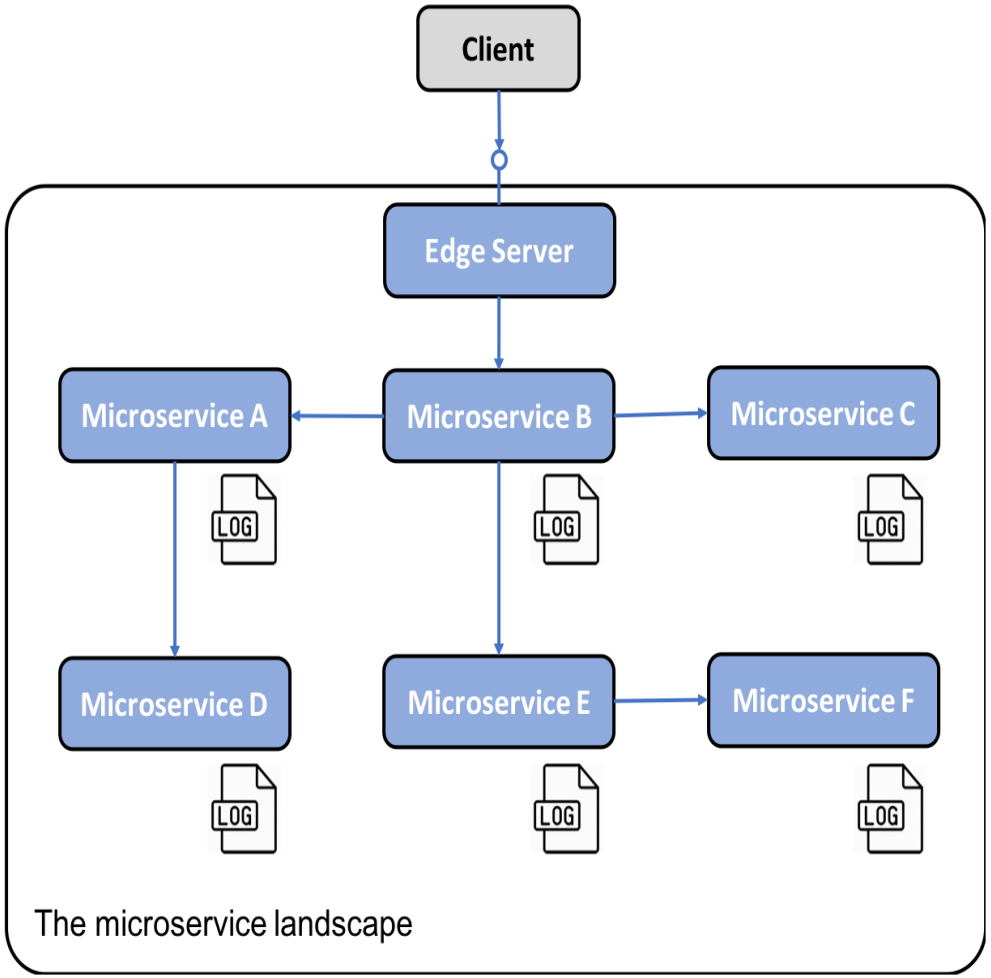
» 로그 분석 중앙화

문제점

- 각 마이크로서비스 인스턴스가 로컬에 로그 파일을 기록하는 상황에서 전체 시스템에서 발생하는 사건을 개괄하려면?
- 문제가 발생한 마이크로서비스 인스턴스를 찾아서 로그 파일에 오류 메시지를 쓰게 하려면?
- 최종 사용자가 문제 발견시, 이와 관련된 로그 메시지를 찾으려면?

해결책

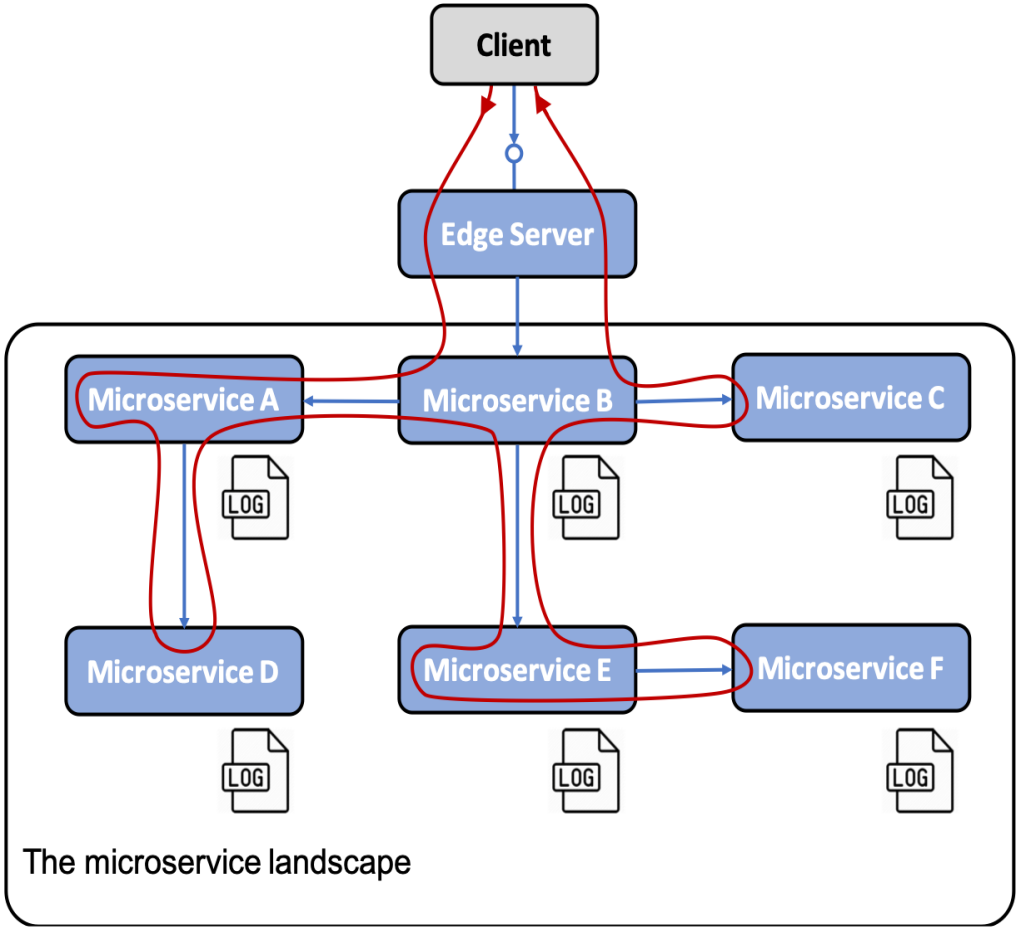
- 마이크로서비스 인스턴스를 감지해 로그이벤트를 수집
- 로그이벤트를 구조적이고 검색가능한 형식으로 중앙 저장소 저장



» 분산 추적

- 문제점
  - 장애 발생시 문제를 일으킨 마이크로서비스를 찾고 근본 원인을 밝히려면?
  - 장애 발생시 이와 연관된 모든 마이크로서비스의 로그 메시지를 찾으려면?

- 해결책
  - 관련요청 및 메시지에 Correlation ID를 넣어야 하고 모든 로그 이벤트에 Correlation ID가 있어야 함



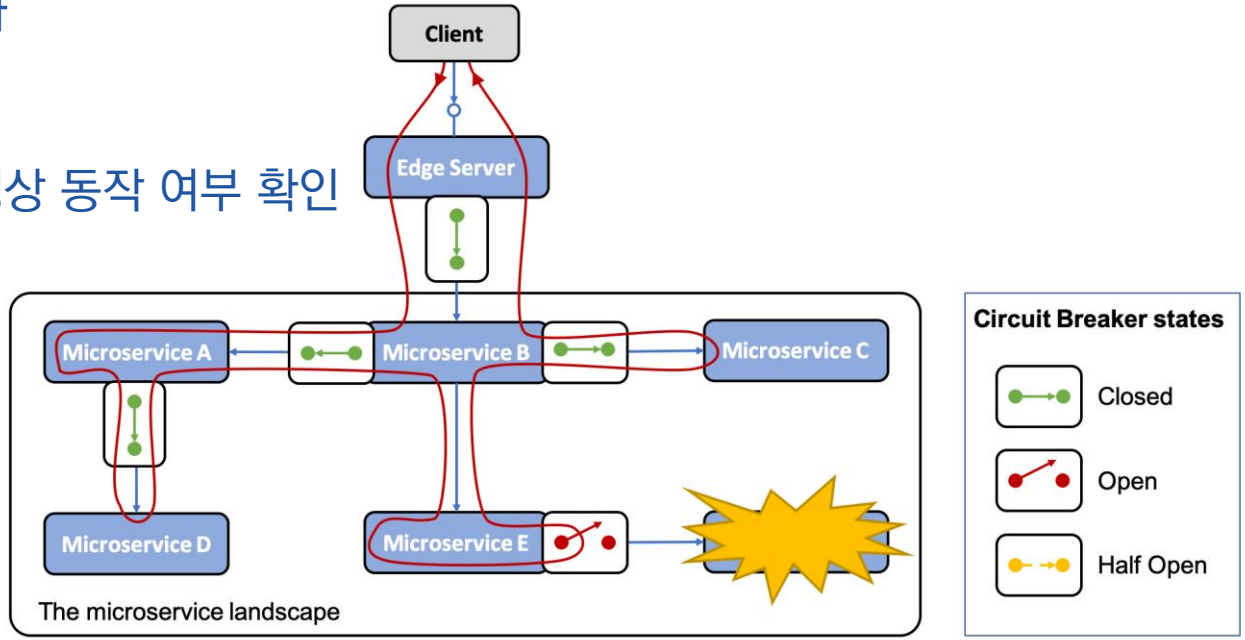
» 서킷 브레이커

문제점

- 동기방식으로 통신하는 마이크로서비스는 연쇄장애 발생가능성 있음

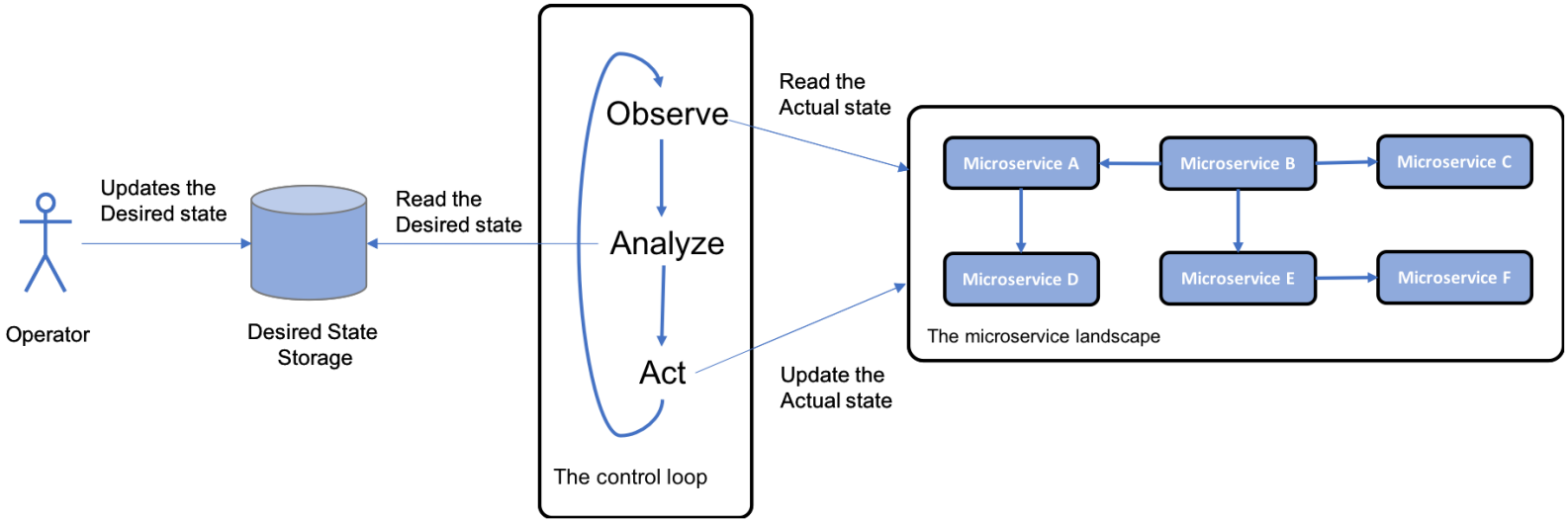
해결책

- 특정 서비스에 문제 발생시 더 이상 새 요청을 그 서비스에 보내지 않도록 차단하는 서킷브레이커를 추가
- 서비스 문제 감지시 서킷 오픈
- 반열림 서킷을 사용. 서비스 정상 동작 여부 확인
- 정상 동작 확인하면 서킷 닫음



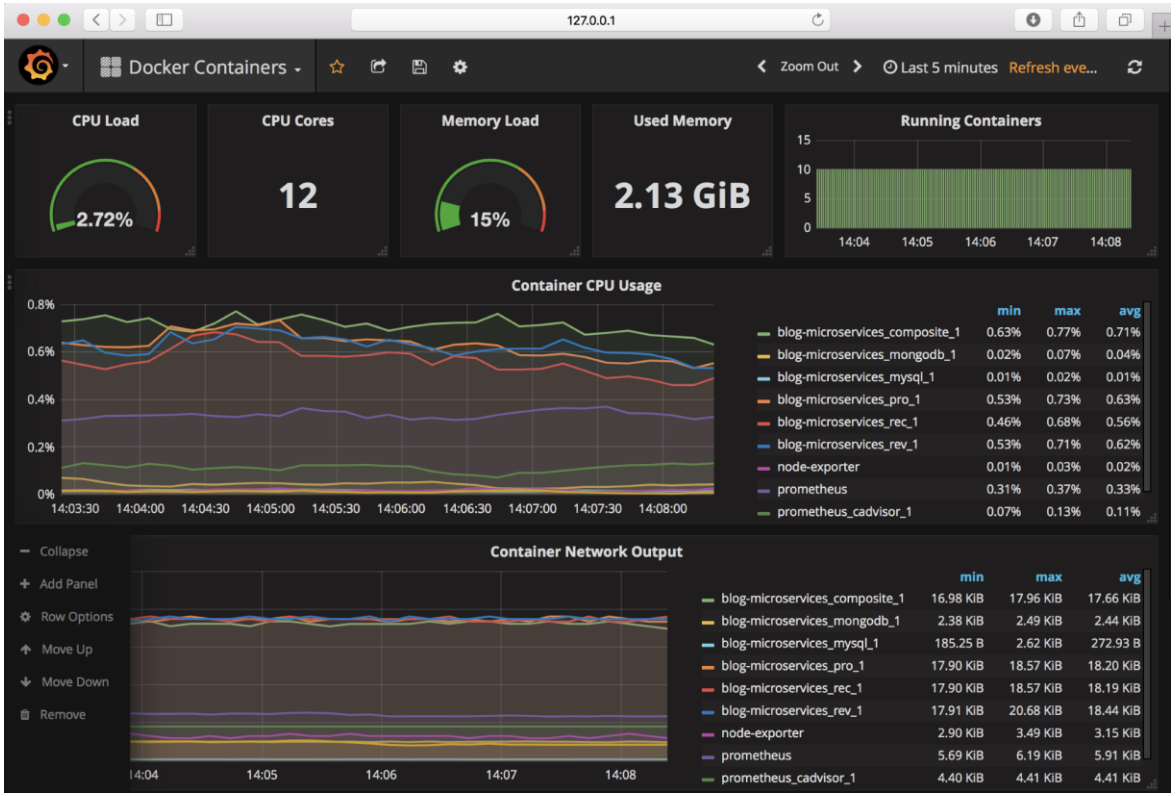
» 제어 루프

- 문제점
  - 다수의 마이크로서비스가 여러 서버에 분산되어 있는 환경에서 마이크로서비스 인스턴스를 수동으로 감지하고 대처하는 것 어려움
- 해결책
  - 시스템 환경 상태를 관찰하는 새 컴포넌트(제어루트)를 시스템환경에 추가함
  - 이 컴포넌트는 운영자가 지정한 상태와 실제 상태를 지속적으로 관찰하여, 두 상태가 일치하도록 조정함



» 모니터링 및 경고 중앙화

- 문제점
  - 응답시간이나 H/W 자원 사용량이 지나치게 높은 경우 문제의 근본 원인 찾기가 어려움.
  - 마이크로서비스별 H/W 자원 사용량 분석이 가능해야 함
- 해결책
  - 마이크로서비스 인스턴스가 사용하는 H/W 자원사용량에 대한 매트릭을 수집하는 컴포넌트를 추가



01 Cloud Native Application이란?

» 클라우드 네이티브 오픈소스 정리

디자인패턴	스프링부트	스프링클라우드	쿠버네티스	이스티오
서비스 검색		유레카, 리본	Kube-proxy, service	
에지서버		Spring cloud G/W, Oauth	인그레스 컨트롤러	이스티오 인그레스 G/W
리액티브 MS	스프링 리액터와 스프링 웹플럭스			
구성중앙화		스프링 컨피그	컨피그맵과 시크릿	
로그분석 중앙화		ELK	ELK, FluentD	
분산 추적		슬루스와 집킨		예거
서킷브레이커		히스트릭스, Resilience4j		이상감지
제어 루프			쿠버네티스 컨트롤러 매니저	
모니터링 및 경고 중앙화			그라파나, 프로메테우스	키알리, 그라파나, 프로메테우스

» Cloud Native Application 개발 원칙

Container 단위	느슨한 결합	독립적/자율적	CI/CD
Stateless	Http, Rest	Bind/분산 DB 지향	DevOps, 자동화

- 12 Factors

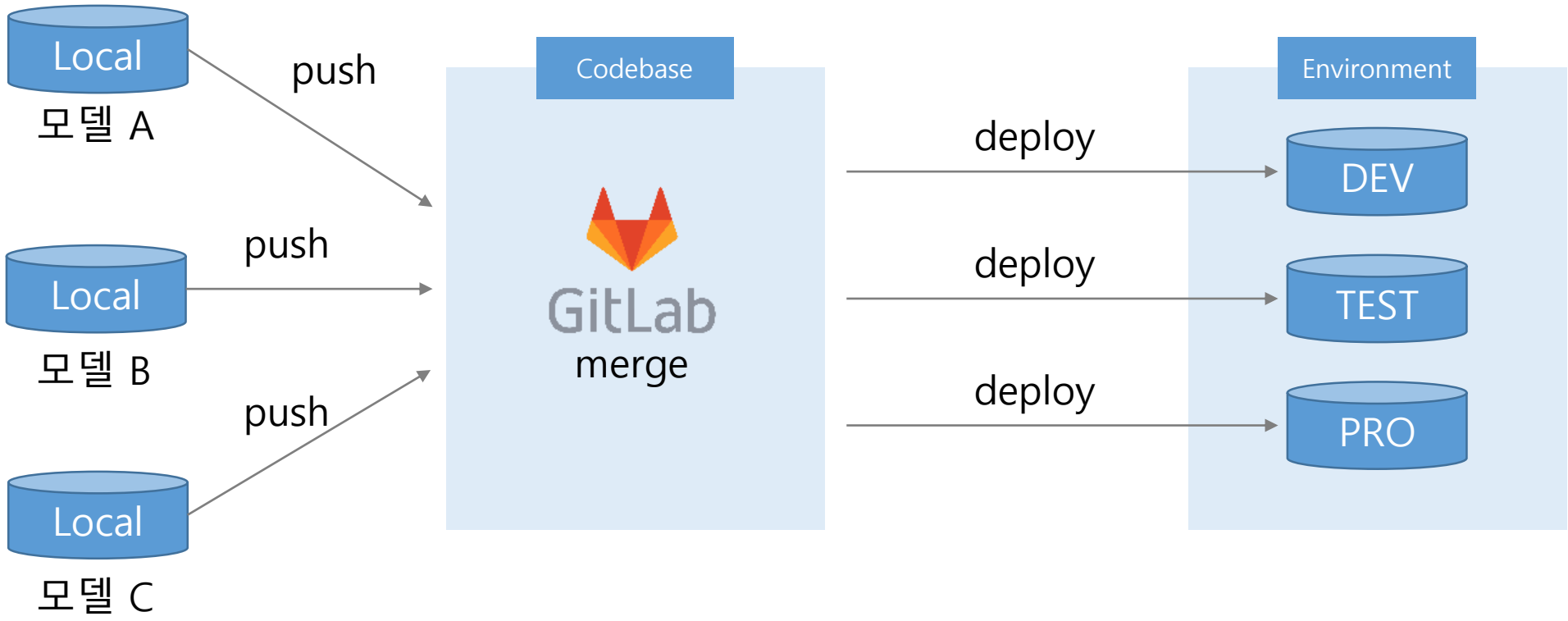
1. Codebase
2. Dependencies
3. Config
4. Backing Services
5. Build, Release, Run
6. Stateless Processes
7. Port Binding
8. Concurrency
9. Disposability
10. Dev/Prod Parity
11. Logs
12. Admin processes

출처 : https://12factor.net/

» Cloud Native Application 개발 원칙

1. 코드 베이스

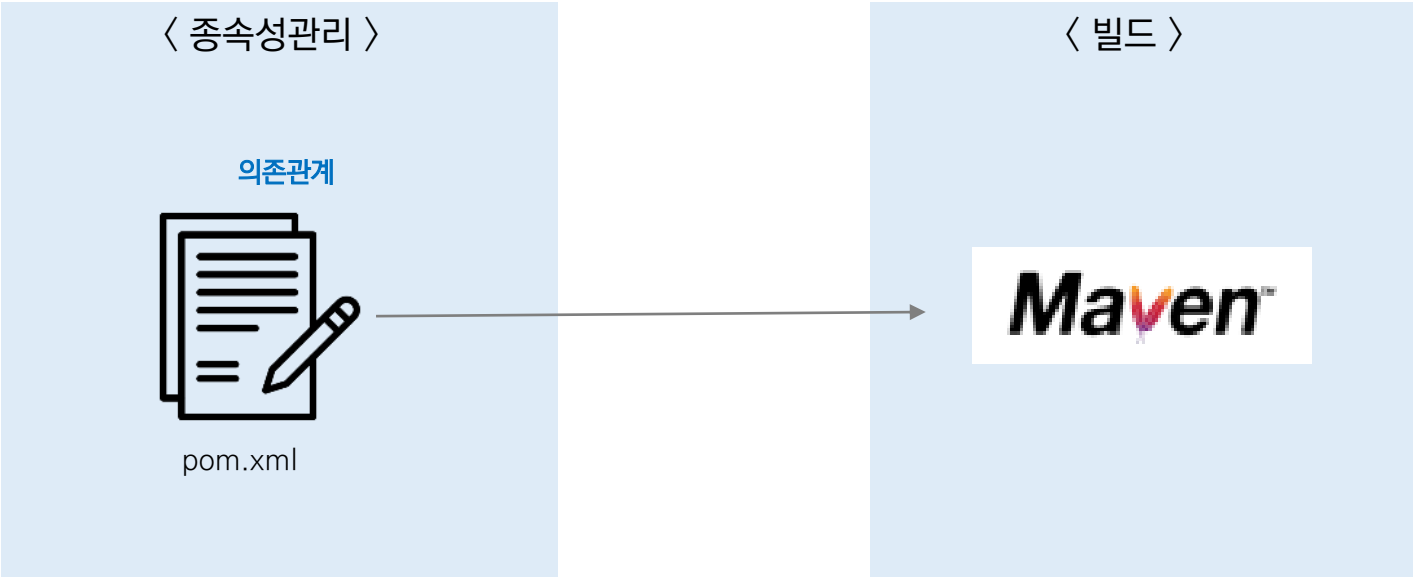
애플리케이션은 **1개의 코드베이스를 통해 관리**되어야 하며, **동일한 코드로 개발 및 운영 환경에 배포**되어야 하며, 코드베이스는 각 애플리케이션마다 단 1개 존재(애플리케이션은 소스코드의 변경과 여러 환경으로 수 차례 배포 시에도 추적 가능함.



» Cloud Native Application 개발 원칙

2. 종속성

종속관계에 있는 모든 애플리케이션은 **명시적으로 종속성을 선언**하며, 애플리케이션이 필요로 하는 **라이브러리를 종속성 파일에 명시적으로 선언**하여 사용

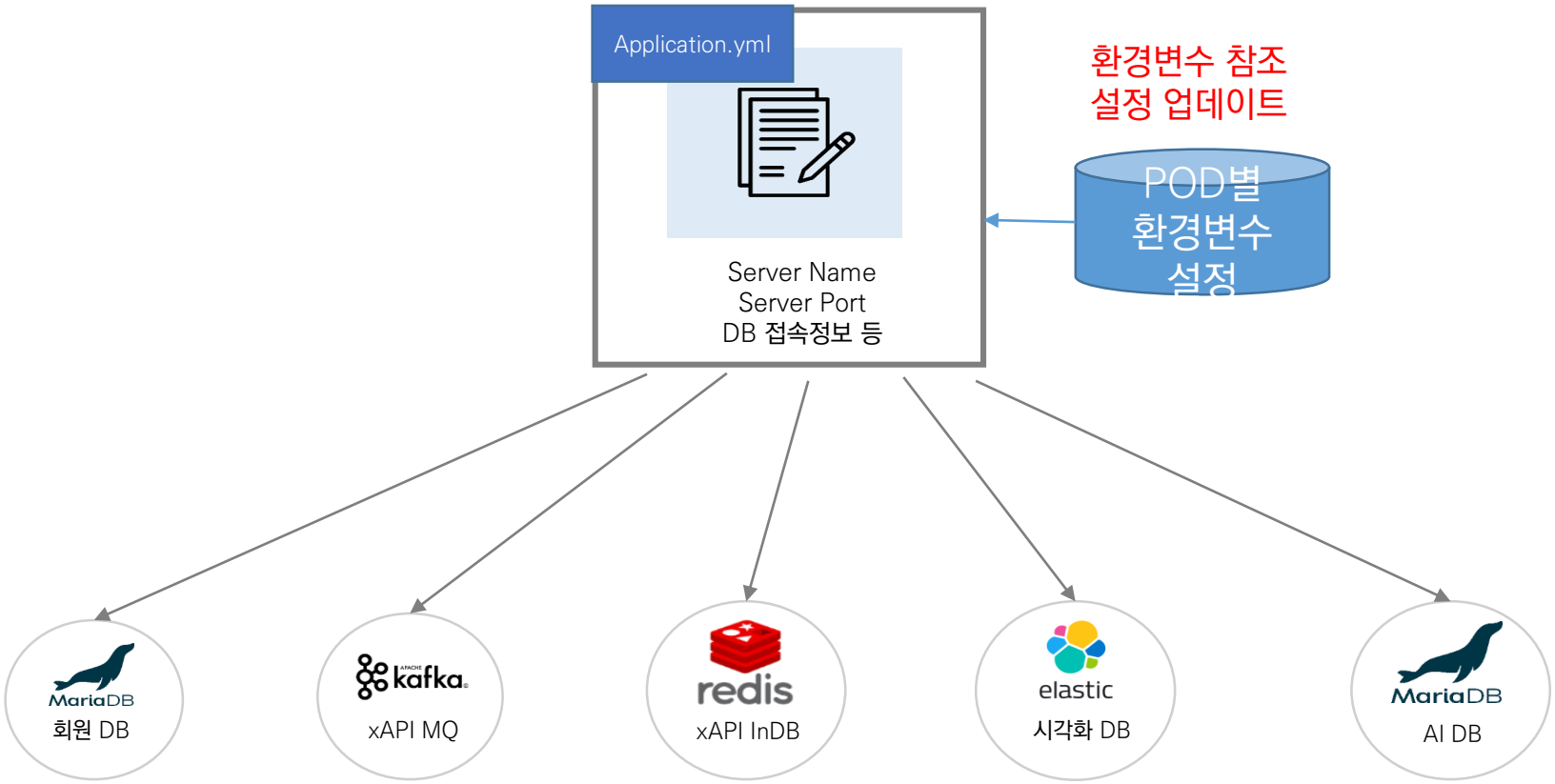


출처 : https://12factor.net/

» Cloud Native Application 개발 원칙

3. 설정

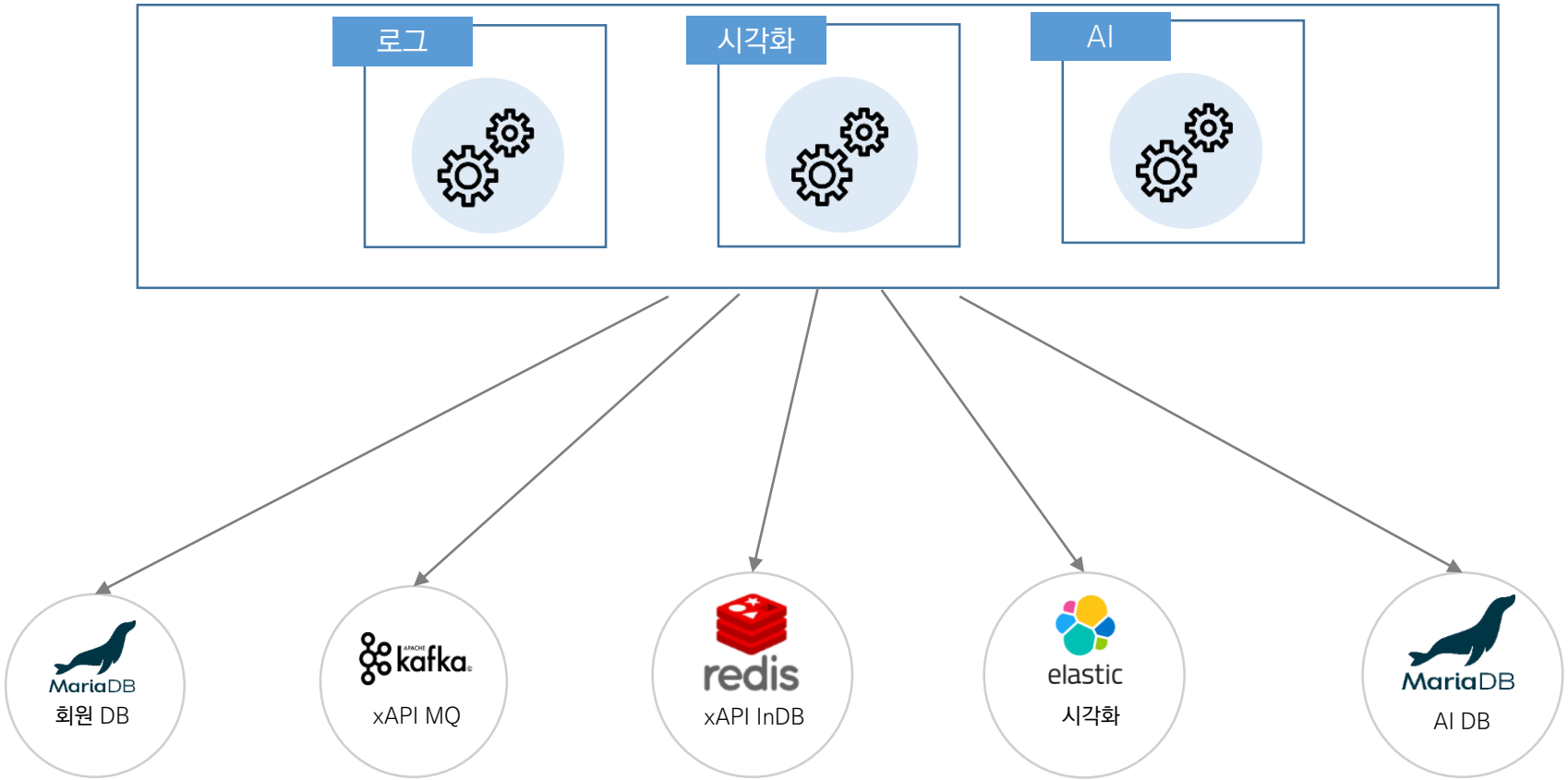
애플리케이션 실행 시 필요로 하는 **설정 정보와 코드는 분리**하여 관리
(업무처리 로직과 무관한 시스템 내외부의 리소스, 배포단계, OS등에 따라 달라지는 설정정보)



» Cloud Native Application 개발 원칙

4. 백엔드 서비스

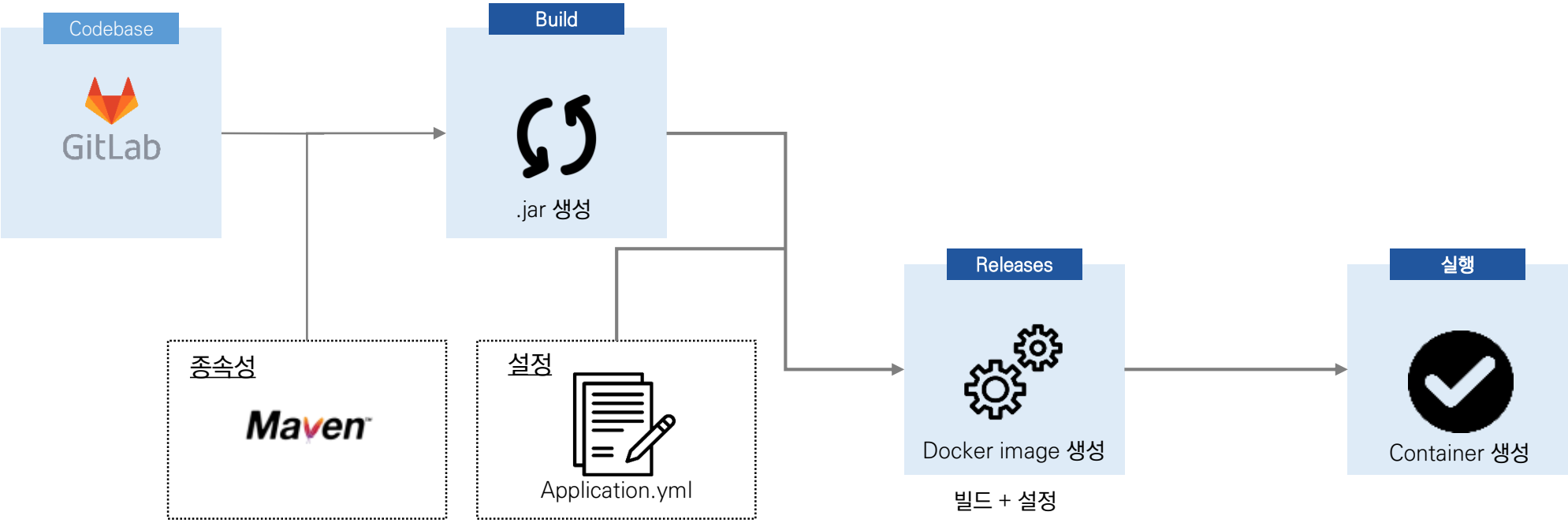
백엔드서비스(DB, 캐시, 메시지/큐, 이메일 서비스, SNS, 클라우드 서비스등)는 **애플리케이션에 연결된 리소스로 취급됨**



» Cloud Native Application 개발 원칙

5. 빌드, 릴리즈, 실행

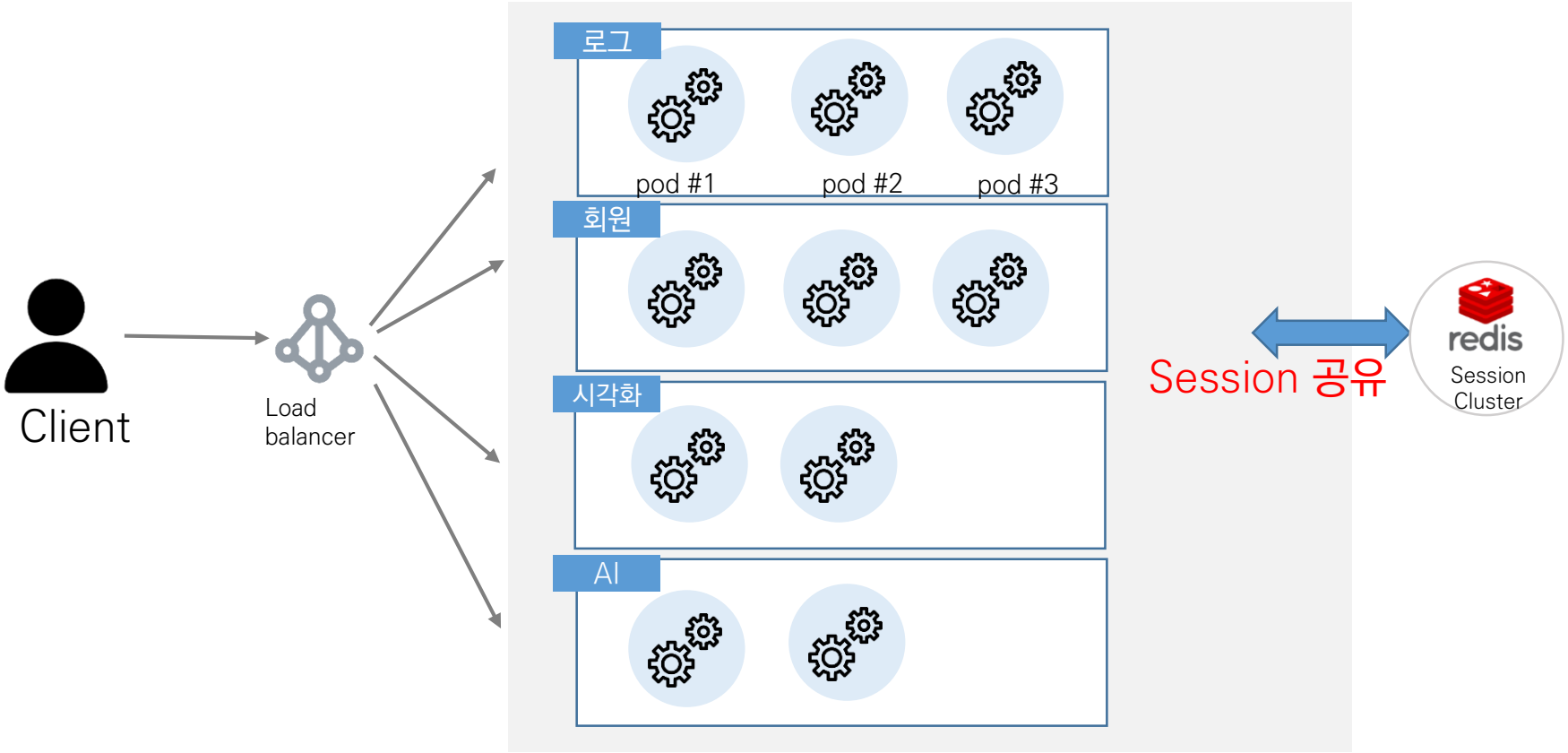
코드베이스는 엄격하게 구분된 빌드, 릴리즈, 실행 3단계의 과정을 통해 배포가 이뤄지며, 각 단계는 엄격하게 분리되어야 함



» Cloud Native Application 개발 원칙

6. 프로세스(무상태 서비스)

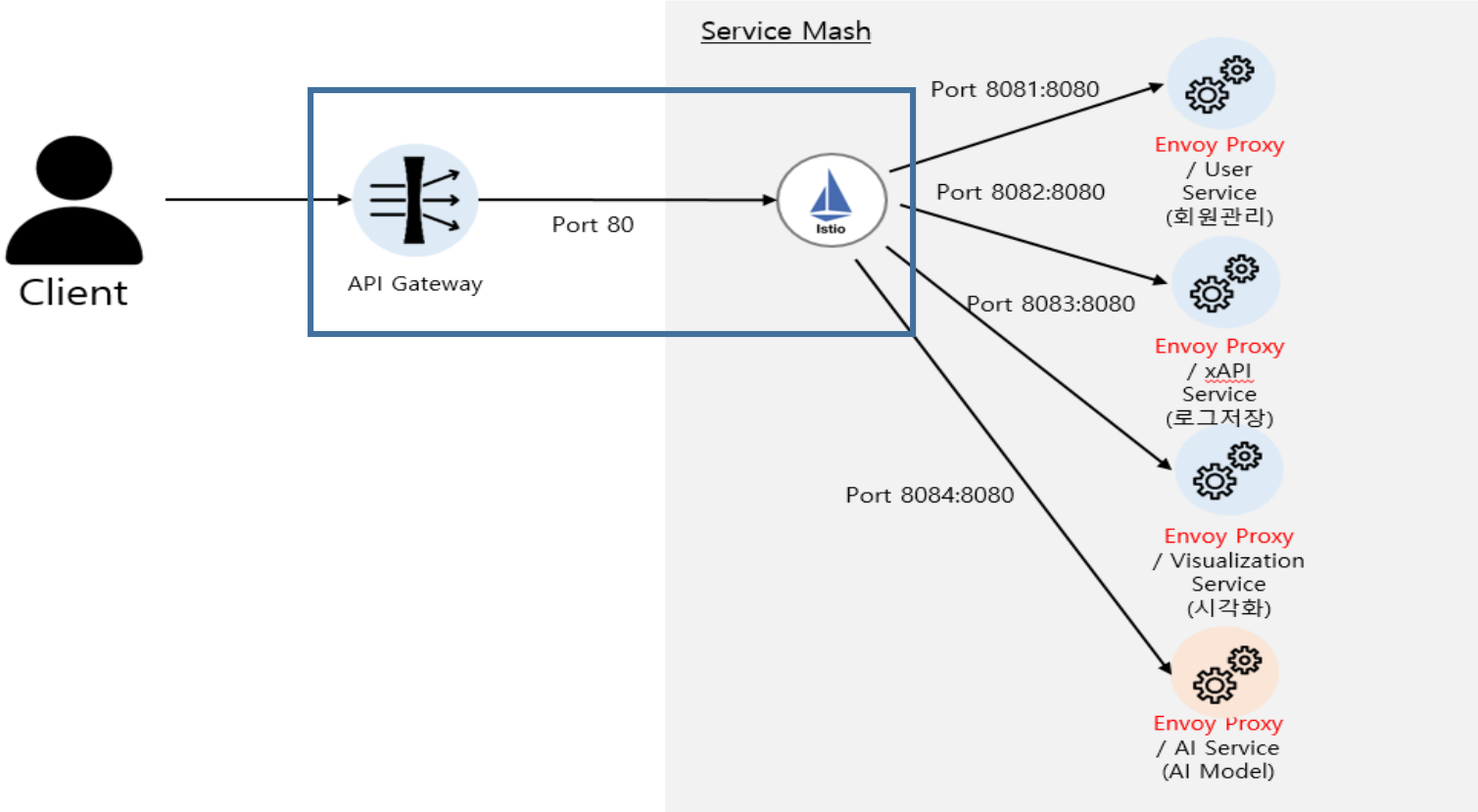
서버에 저장하지 않고 외부에 저장하여 스케일아웃이 용이한 구조로 구성



» Cloud Native Application 개발 원칙

7. 포트 바인딩

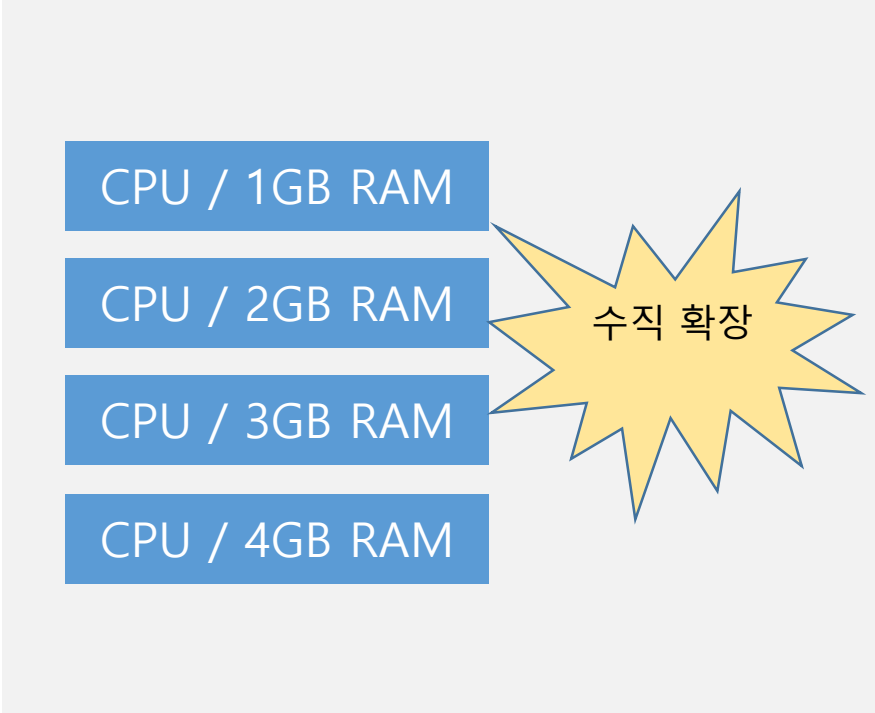
메시지를 송수신하는 위치와 방법을 결정하는 구성정보를 의미하며, 배포된 애플리케이션을 타 애플리케이션에서 접근할 수 있도록 포트 바인딩을 통해 서비스를 공개



» Cloud Native Application 개발 원칙

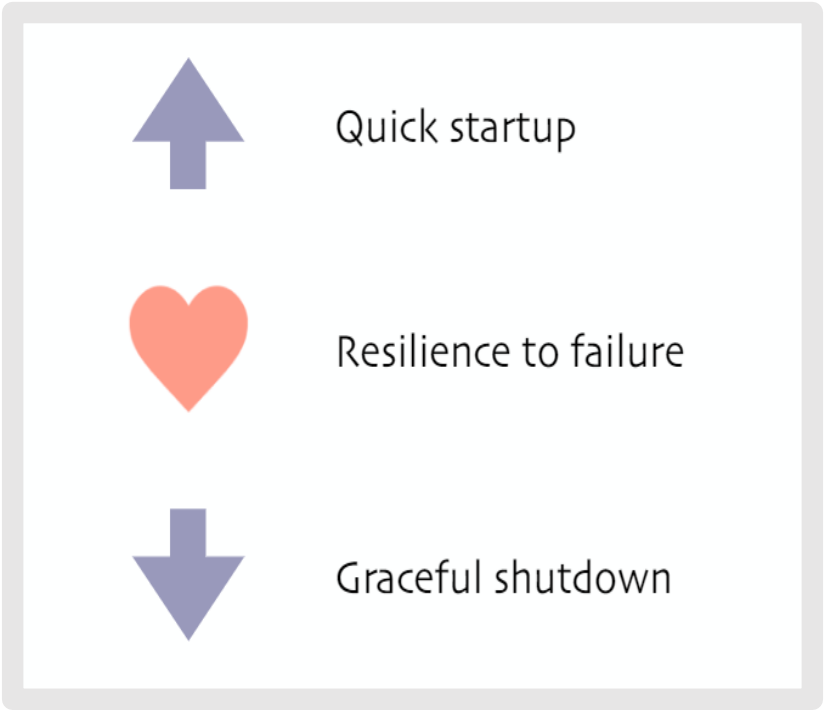
8. 동시성

모든 일을 처리하는 하나의 프로세스 대신 기능별로 분리된 마이크로 프로세스를 실행하며, 프로세스 모델을 기반으로 수직적(Scale-up) 및 수평적(Scale-out) 확장성을 제공



» Cloud Native Application 개발 원칙

9. 폐기 가능



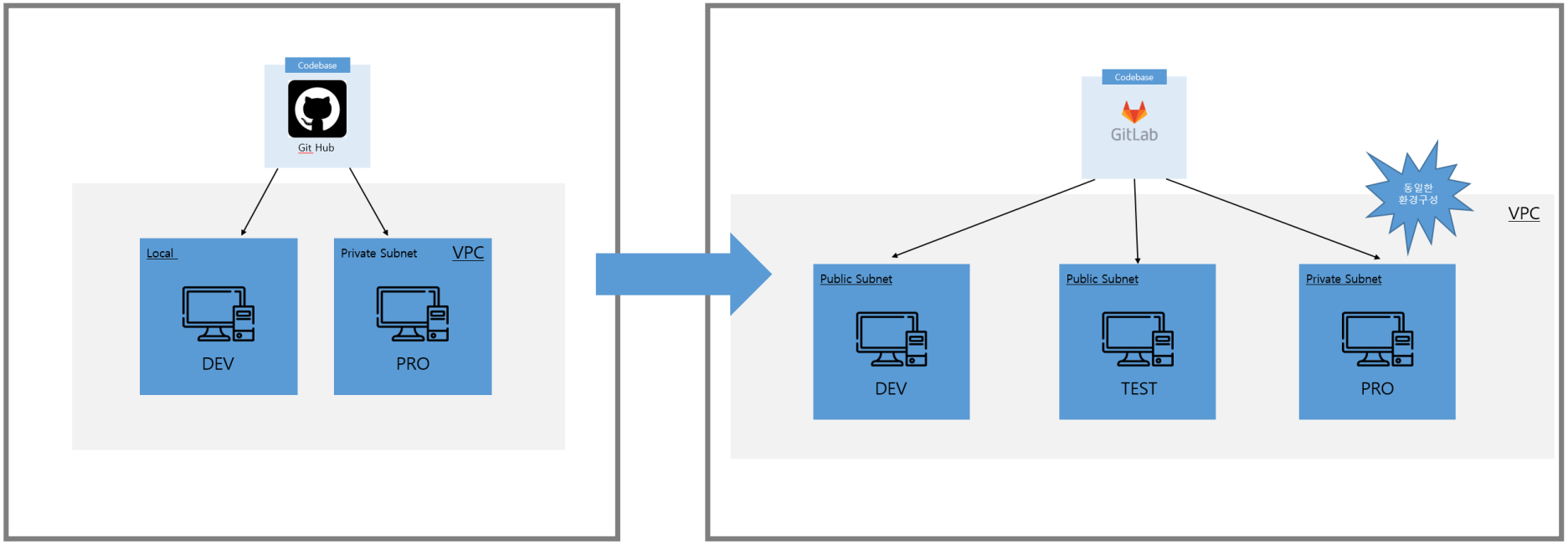
빠른 시작과 그레이스풀 셧다운을 통한 안정성 극대화

- 프로세스는 바로 시작하거나 종료 할 수 있도록 설계
- 시작 시간을 최소화하여 릴리즈 작업과 확장이 민첩하게 이루어질 수 있음
- 그레이스풀 셧다운을 통해 처리중인 작업을 큐로 되돌리는 방법으로 구현
- 갑작스러운 셧다운에도 작업을 큐로 되돌릴 수 있도록 진행

» Cloud Native Application 개발 원칙

10. DEV / PROD 일치

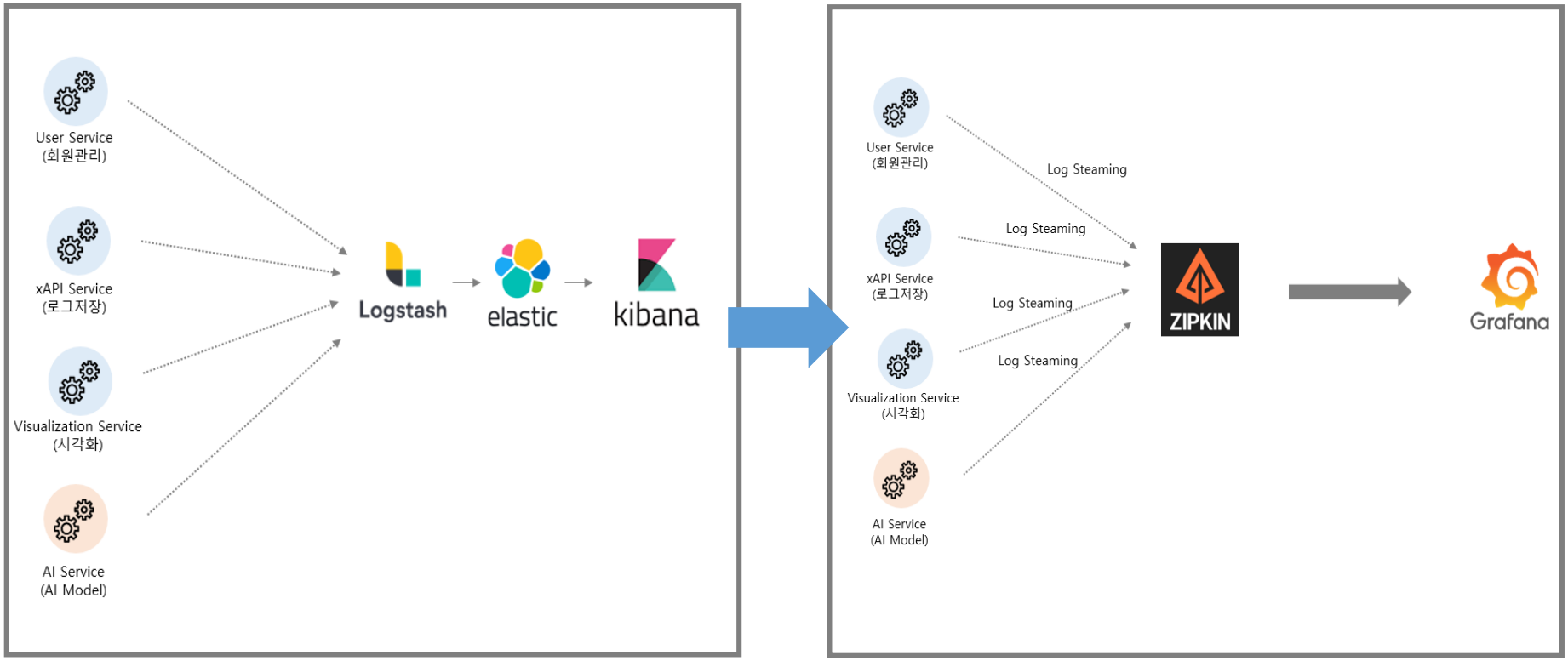
가능한 동일한 개발/검증/운영 환경을 유지하여 장애 최소화 및 지속적 배포가 가능



» Cloud Native Application 개발 원칙

11. 로그

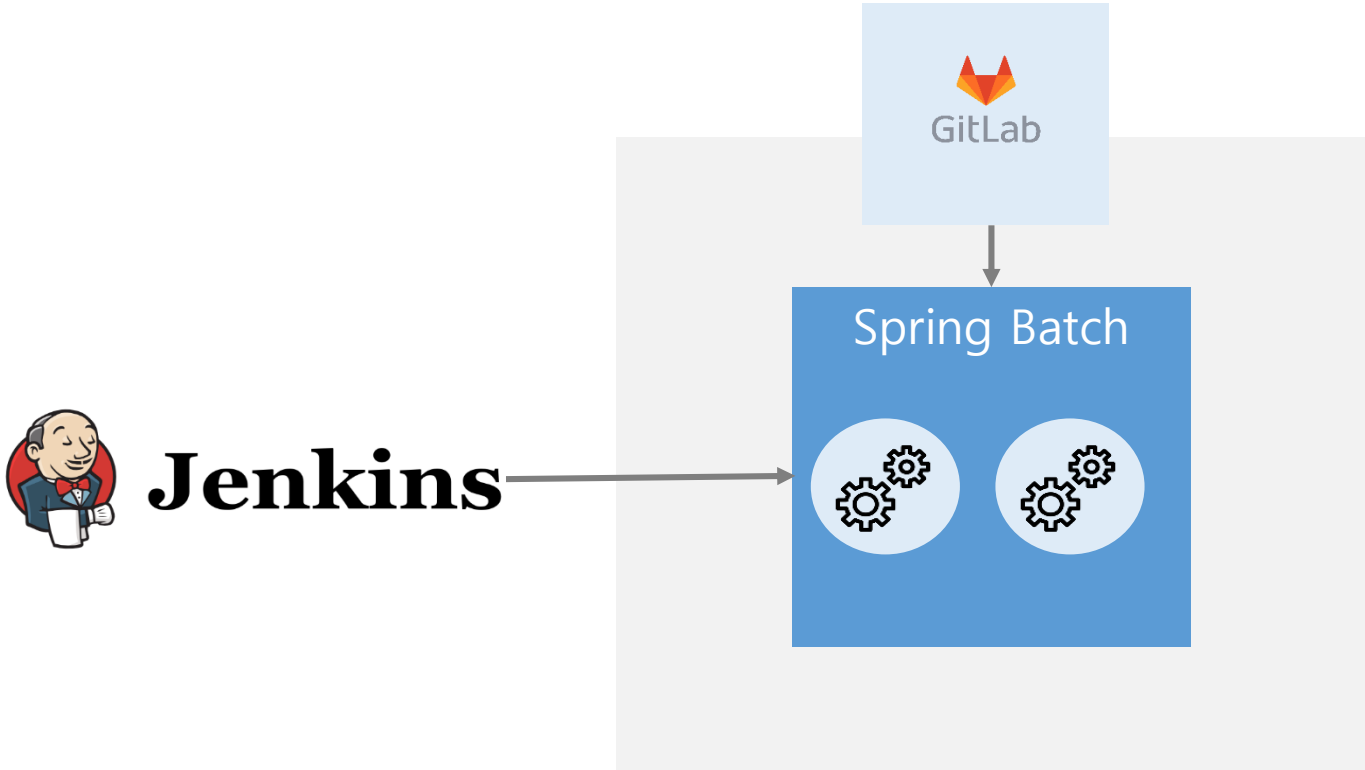
어플리케이션 로그는 파일이 아닌 로그 스트림 형태로 표준 출력하며, 플랫폼에 의해 로그 스트림이 통합되고 관리됨



» Cloud Native Application 개발 원칙

12. Admin 프로세스

관리/유지보수 프로세스를 일회성 프로세스로 실행하며, 관리/유지보수 프로세스는 일회성 프로세스로 어플리케이션 프로세스와 분리되어 실행되지만, 어플리케이션과 동일한 빌드/릴리즈/배포 사이클로 실행



02 왜 Cloud Native Application을 이용하는가?

» Cloud Native Application 사용 이유

Speed of Innovation

- 다양한 신기술을 적용하여 애플리케이션 제공 또는 새 버전의 소프트웨어 배포
- 잘못 배포했을 경우 즉각적인 복구 필요

Always-Available Services(Safety)

- 장애 영향을 받는 서비스 구성범위를 제한하고, 퍼지지 않도록 의존성 최소화
- 정형화 된 유형의 장애들은 시스템이 자동적으로 복구

Web Scale

- Scale up을 통한 수직적인 확장 → 비용부담이 적고 빠른 Scale Out을 통한 수평적인 확장
- 작고 균일한 서버들의 가상화를 통한 Workload Orchestration

Mobile-Centric User Experiences

- IT 환경과 비즈니스 서비스에서 다양성의 극단적인 확대
- 스마트폰 저변 확대에 따른 Mobile 중심의 개발 필요성 대두

왜 Cloud Native Application을 이용하는가?

» 기존 애플리케이션의 문제점 대두

- 규모가 큰 애플리케이션에서는 불리
  - 빌드, 배포, 서버 기동 시 시간이 오래 걸림
  - 한 두 사람의 실수는 전체 시스템의 빌드 실패를 유발하여 프로젝트가 커질수록 협업하기 어려움
  - 컴포넌트들이 서로 로컬 콜 (Call-By-Reference)기반으로 강하게 결합(Tightly Coupled)되어 특정 컴포넌트나 모듈에서의 성능 문제나 장애가 다른 컴포넌트에까지 영향
  - 코드가 너무 커져서 유지보수 어려움
  - 기존 로직, 데이터, 인터페이스 등의 변경에 따른 영향을 파악하기 어려움
- 배포가 잦은 시스템에 불리
  - 사소한 컴포넌트의 수정도 전체 애플리케이션을 재컴파일하여 배포하고, QA를 거쳐야 함

왜 Cloud Native Application을 이용하는가?

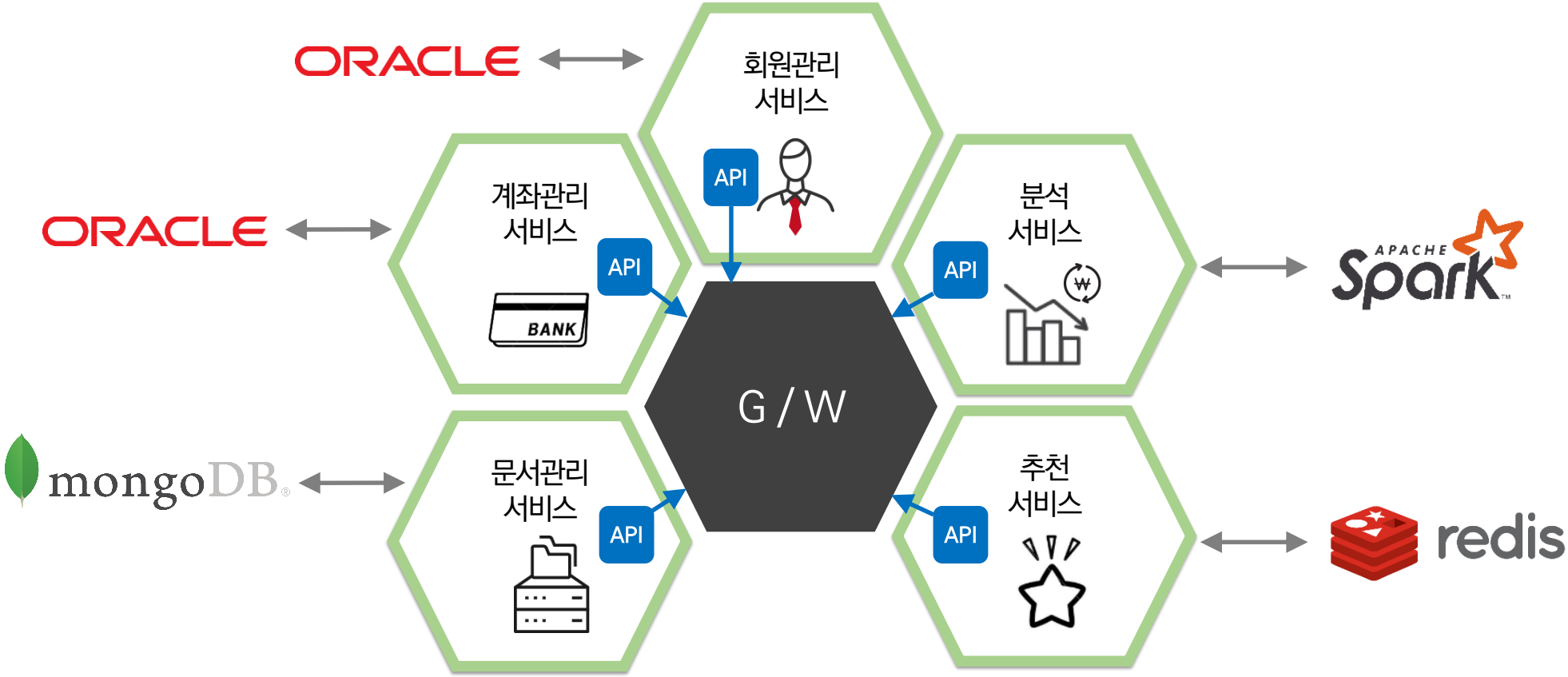
» 기존 애플리케이션의 문제점 대두

- 새로운 기술에 대한 도입 어려움
  - 컴포넌트 별로 기능/비기능적 특성에 맞춰서 다른 기술을 도입하고자 할 때 유연하지 않음

    예) 파일 업로드/다운 로드와 같이 IO 작업이 많은 컴포넌트의 경우 node.js를 사용하는 것이 좋을 수 있으나, 애플리케이션이 자바로 개발되면 다른 기술 추가가 매우 어려움
  - On-Premise Cloud, CI와 연계된 배포 자동화(Jarvis), 향후 Docker와 같은 Container 기술과 연계 어려움
- 경직성
  - 시스템을 분리, DB의 분리 어려움
  - 시스템간 연계의 증대에 대한 유연한 대응이 어려움
  - 새로운 버전이나 기술의 도입 어려움 또는 불가능
  - 조직의 비대화, 코드의 비대화로 인한 변화에 대한 저항 또는 장벽으로 작용
  - 한 애플리케이션에서 개발한 기능은 타 애플리케이션에서 사용하기 어려움

왜 Cloud Native Application을 이용하는가?

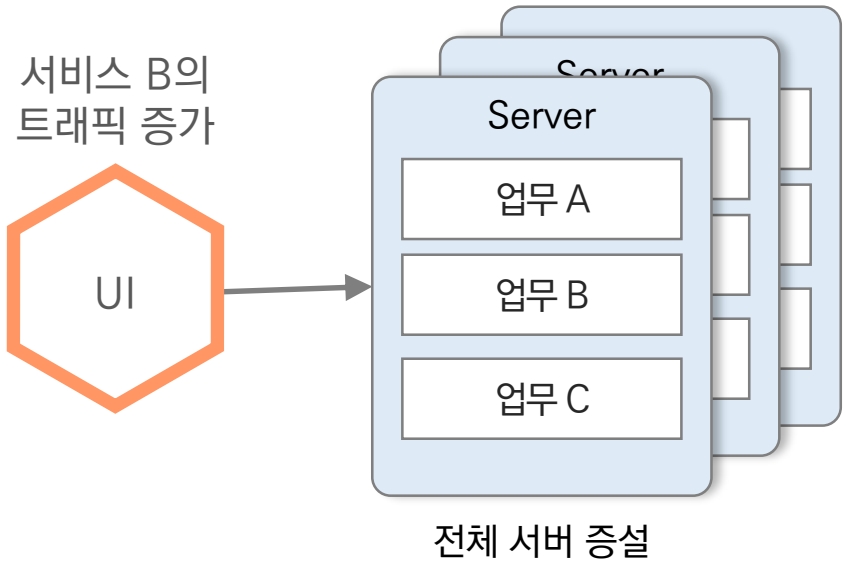
» 다양한 기술 적용

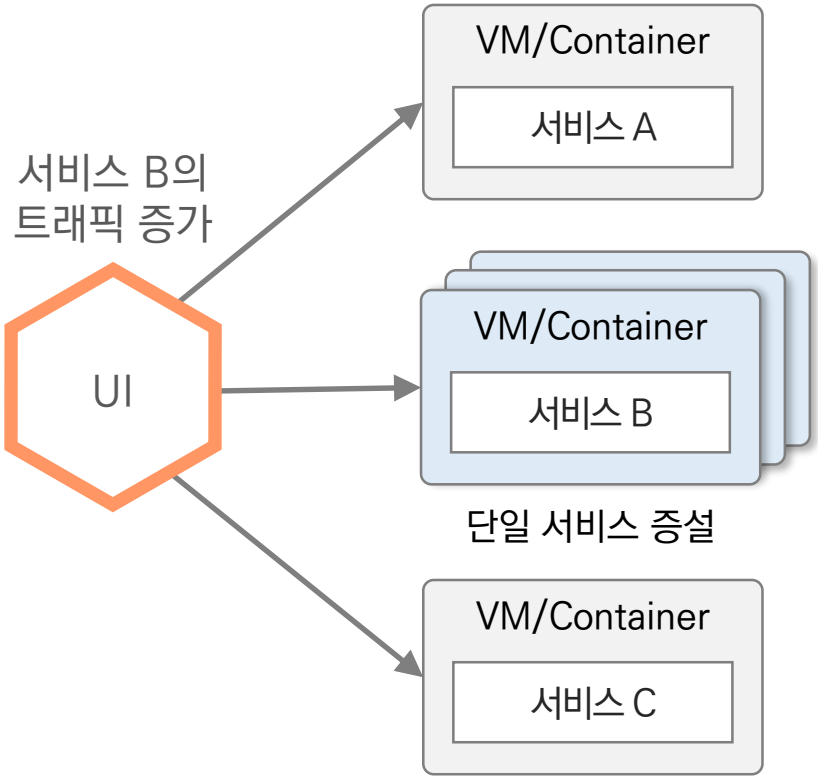


왜 Cloud Native Application을 이용하는가?

» 선택적 확장

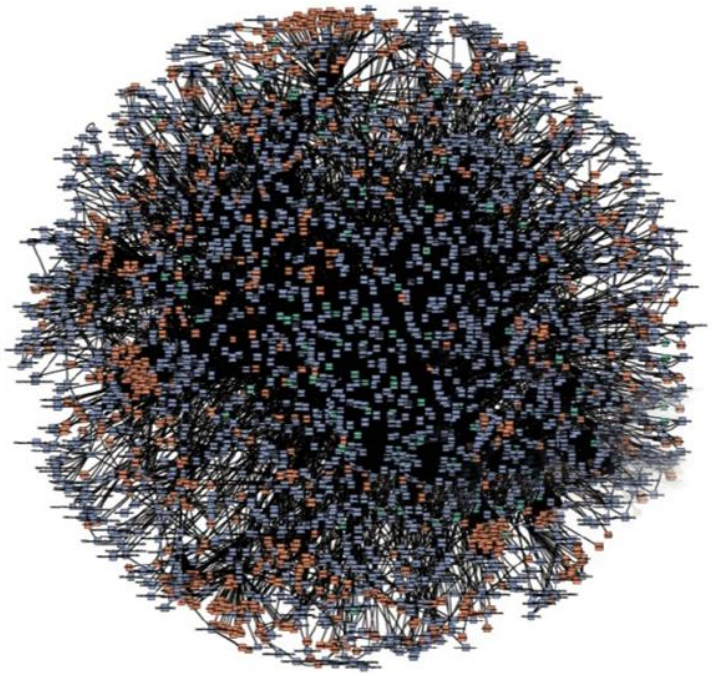
전통적인 아키텍처	VS	클라우드 네이티브 아키텍처
✔ 특정 업무의 트래픽 증가 시에도 전체 서버 단위로 증설		✔ 트래픽이 증가한 서비스의 컨테이너만 유연하게 확장과 축소





왜 Cloud Native Application을 이용하는가?

» 가볍고 손쉬운 배포방식의 필요



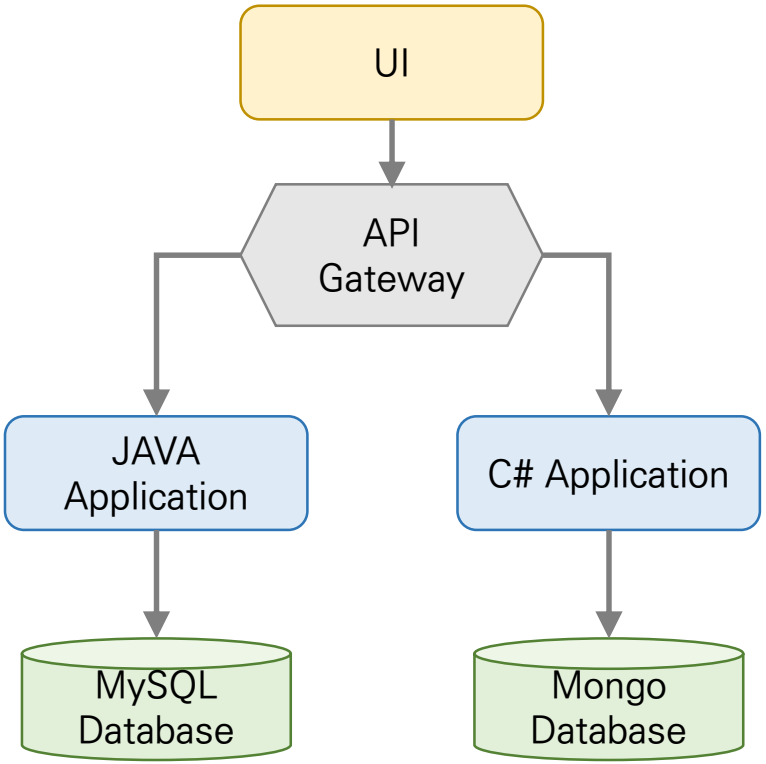
수 천개 팀 (자율적 DevOps팀)
× 마이크로서비스 아키텍처
× 지속적 배포(CD)
×다양한 개발 환경

= **연간 5천만회 배포**(2014년)
(시간당 5708회, 또는 0.63초)

출처 : Werner Vogels (CTO, Amazon.com, 2015)

» Cloud Native Application 장점

장점(1/5)

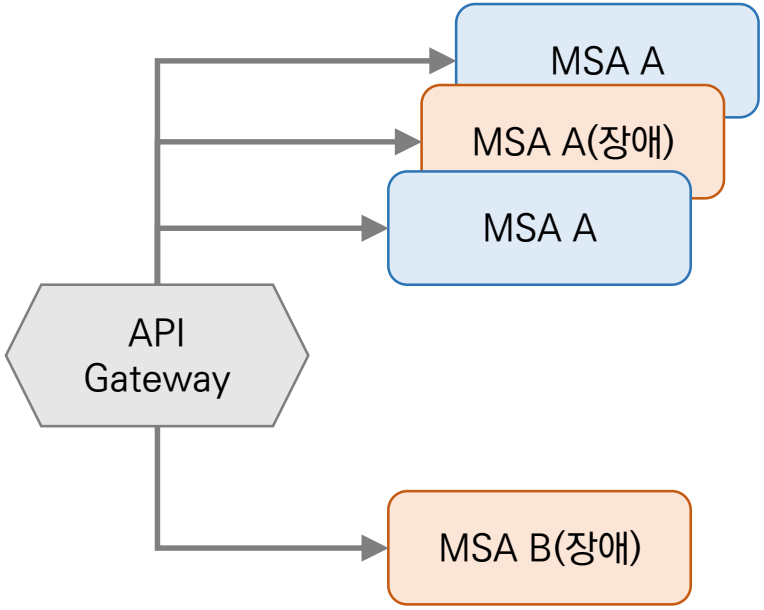


최적화된 언어와 아키텍처 선택

- Cloud Native Application을 이루고 있는 각각의 Micro Service는 독립적으로 운영
- Service 간의 호출은 HTTP Restful API를 통한 느슨한 결합
- 각 시스템의 프로그래밍 언어가 다르거나 이기종의 Database를 사용하여도 API 규약에 따른 통신가능
- 각 서비스에 최적화된 언어와 아키텍처 구성 가능

» Cloud Native Application 장점

장점(2/5)

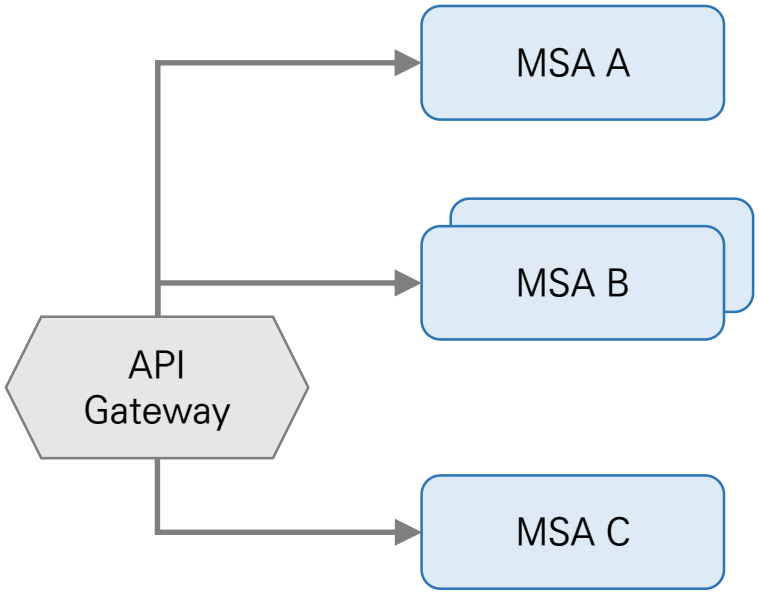


탄력적인 시스템 운영

- 장애 발생 시 복구될 때까지 요청 기능 서비스에서 제외하여 무의미한 커넥션 방지
- 수평적 확장된 서비스에서 장애 시 장애 발생 시 로드밸런서를 통해 장애 발생 된 인스턴스를 제외하여 서비스 정상 운영

» Cloud Native Application 장점

장점(3/5)

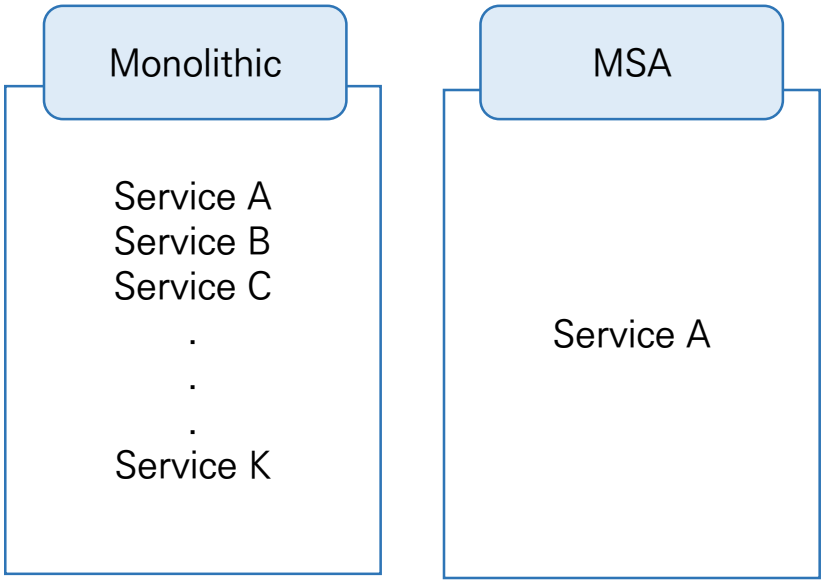


독립적 확장성

- 서비스들은 서로 독립적이므로 타 서비스에 영향을 주지 않고 서비스 단위로 확장 가능
- X축 확장으로 불리는 멀티 애플리케이션 확장과 Z축 확장(Partitioning 또는 Sharding)을 독립적으로 수행
- 전체 시스템을 확장하여야 하는 모놀리식(Monolithic) 방식에 비해 적은 리소스로 시스템 확장 가능

» Cloud Native Application 장점

장점(4/5)

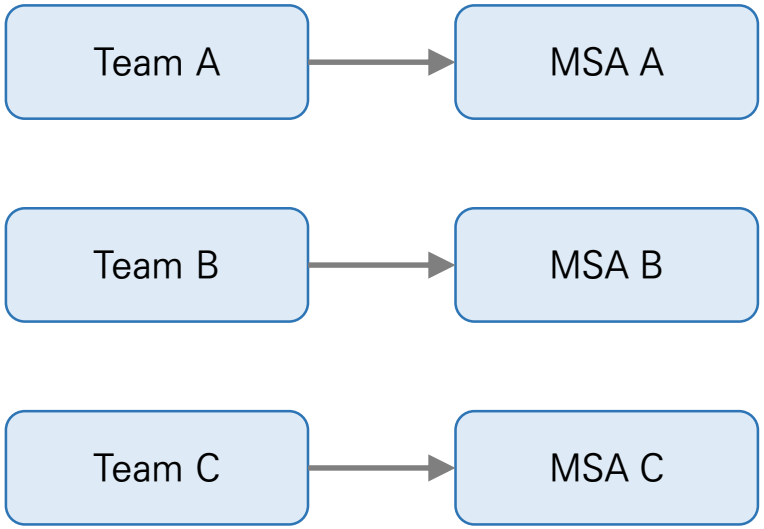


손쉬운 개발

- 각각의 마이크로 서비스는 단일 기능만 수행하는 단순한 구조로 개발 속도와 개선에 높은 효율성 보유
- 클라우드에서 제공하는 하드웨어와 소프트웨어를 이용하여 빠른 구축 가능
- 자동화된 단위 테스트와 시나리오 테스트를 통해 빠른 배포 주기에도 불구 뛰어난 품질 유지

» Cloud Native Application 장점

장점(5/5)

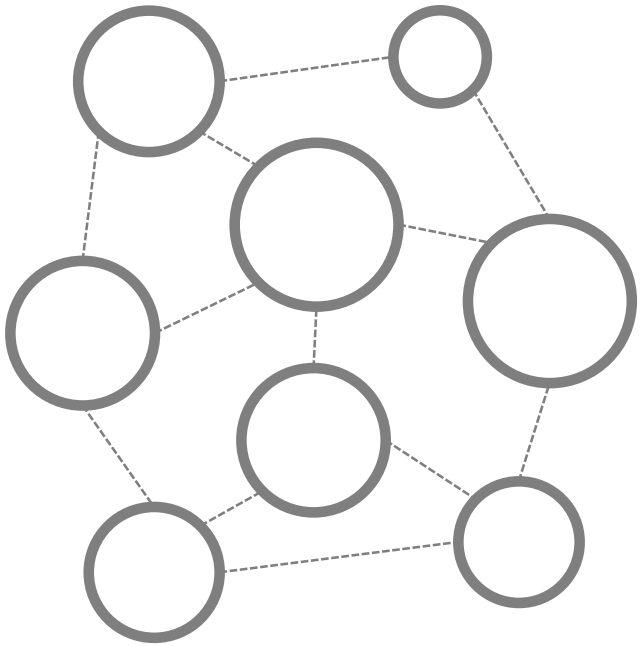


스크럼 구성을 통한 배포

- 각각의 마이크로서비스는 개별 팀에서 독립적으로 개발 및 배포가 가능
- 시스템 규모가 커짐에 따라 추가로 발생하게 되는 오버헤드를 일정 수준으로 관리 가능

» Cloud Native Application 단점

단점(1/3)

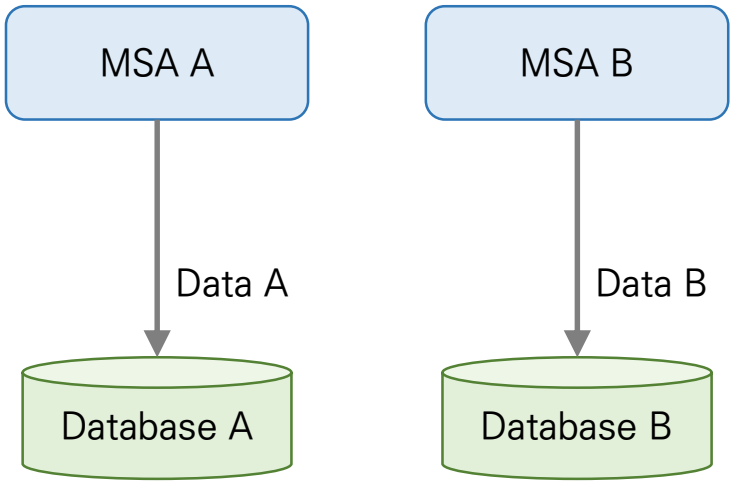


복잡성 증가

- 시스템의 각 서비스들이 마이크로서비스로 분산되어 관리 포인트의 증가
- 전체 시스템에 대한 아키텍처 복잡도가 증가하여 전체적인 구조 파악의 어려움
- 분산 시스템 추가 시 네트워크 대기 시간, 내결함성,직렬화 등 고려해야 할 요소 증가

» Cloud Native Application 단점

단점(2/3)

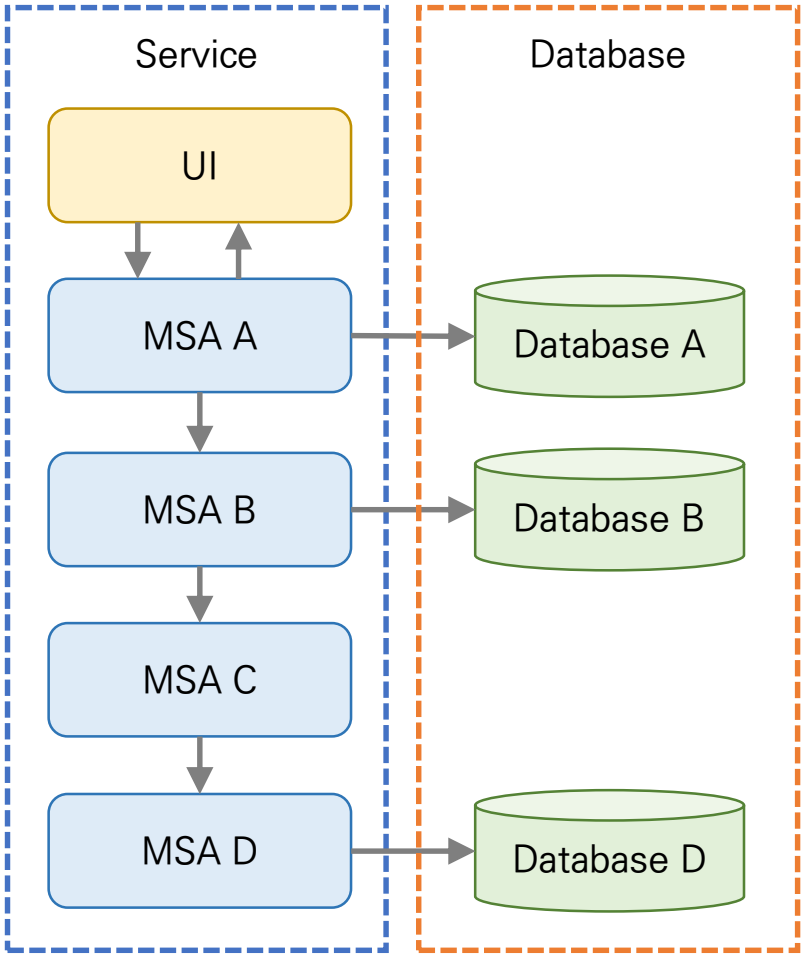


데이터 중복성 증가

- 분산 데이터베이스로 인하여 중복 데이터 저장필요 (반정규화)
- 각 데이터베이스에 각자 데이터가 저장되므로 데이터 무결성이 깨질 가능성 증가
- 멀티 데이터베이스에 트랜잭션 처리가 필요할 수 있어 트랜잭션 관리에 어려움을 겪을 가능성 증가

» Cloud Native Application 단점

단점(3/3)



종단간 테스트의 어려움

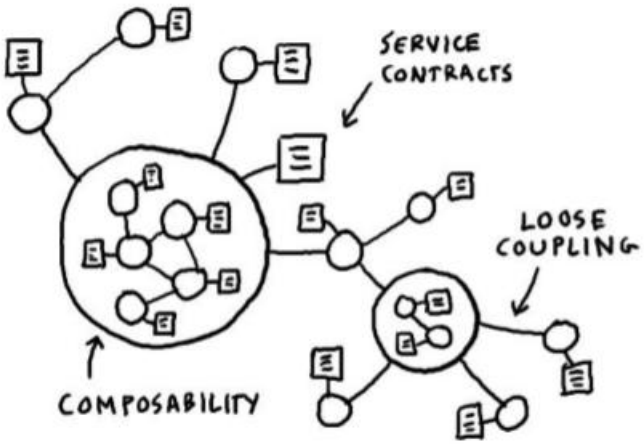
- 마이크로 서비스의 구조상 단일 비즈니스 로직 처리를 위하여 다수의 마이크로서비스 연계 가능
- 예외 사항발생 시 이용되는 MSA 노드들을 모두 추적하여 에러 케이스 확인 필요.
- 각 서비스의 개발 및 운영 담당자가 달라 에러 추적의 어려움 발생

03. MSA(Micro Service Architecture)

» MSA 개요

MSA 개요

- One Thing
  한가지 기능(비즈니스 관련 기능 및 역할)을 수행하는데 초점을 맞춘 서비스
- Small
  독립적이고 배포 가능한 가장 작은 단위의 서비스로 분리
- API
  API를 통해 다른 서비스와 연계
- Autonomous
  각자 자율적으로 개발, 운영. 즉, 독립적인 팀이 각 서비스의 개발과 운영을 담당



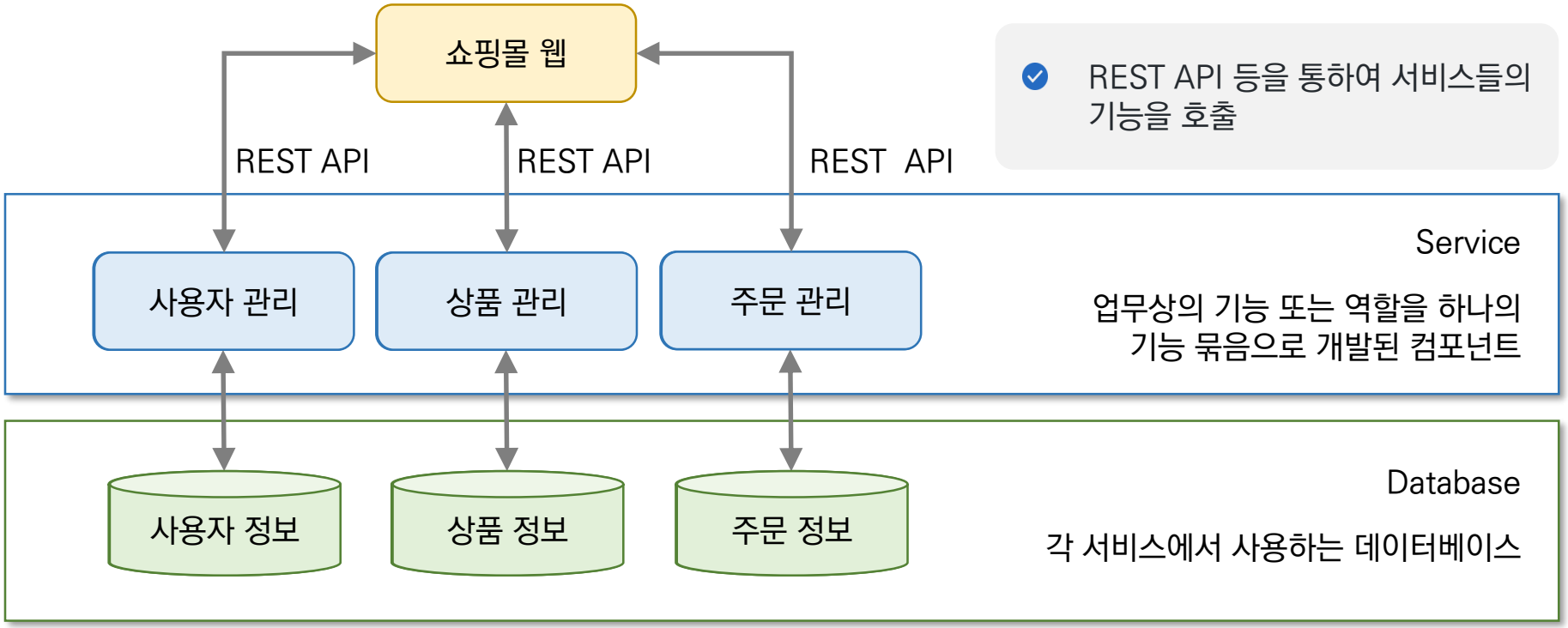
"마이크로서비스 아키텍처 스타일이란 단일 애플리케이션을 자체 프로세스로 실행되고 경량 매커니즘(주로 HTTP 리소스 API)으로 통신하는 작은 서비스들의 모음으로 개발하는 방식이다"

– 마틴 파울러(Martin Fowler)

» MicroService Architecture 예시

MicroService Architecture 예시

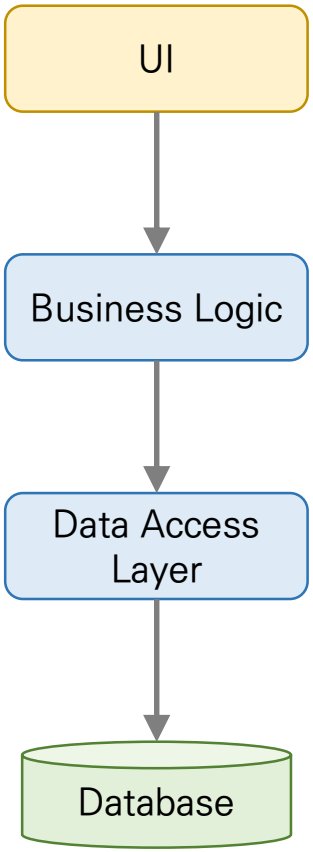
- 하나의 기능을 구현 하는데 여러 개의 서비스를 조합하여 기능을 제공
  예) 주문 하기 : 사용자 정보 조회 , 상품 정보 조회 , 신규 주문 생성

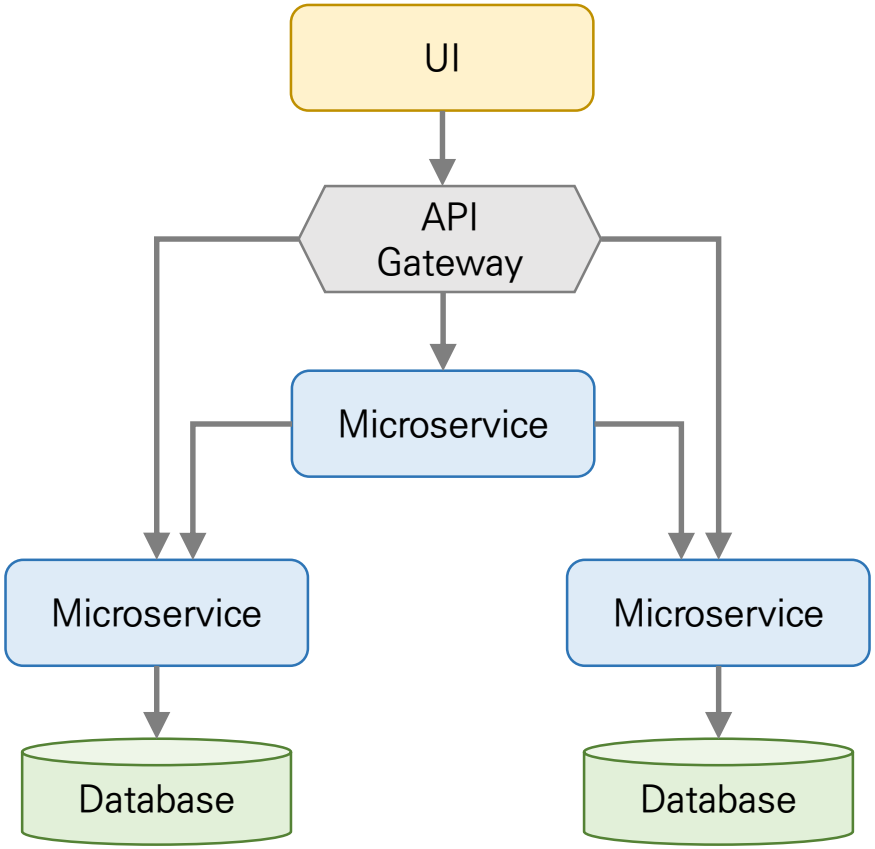


- REST API 등을 통하여 서비스들의 기능을 호출

» Monolithic VS MicroService

Monolithic	VS	MicroService





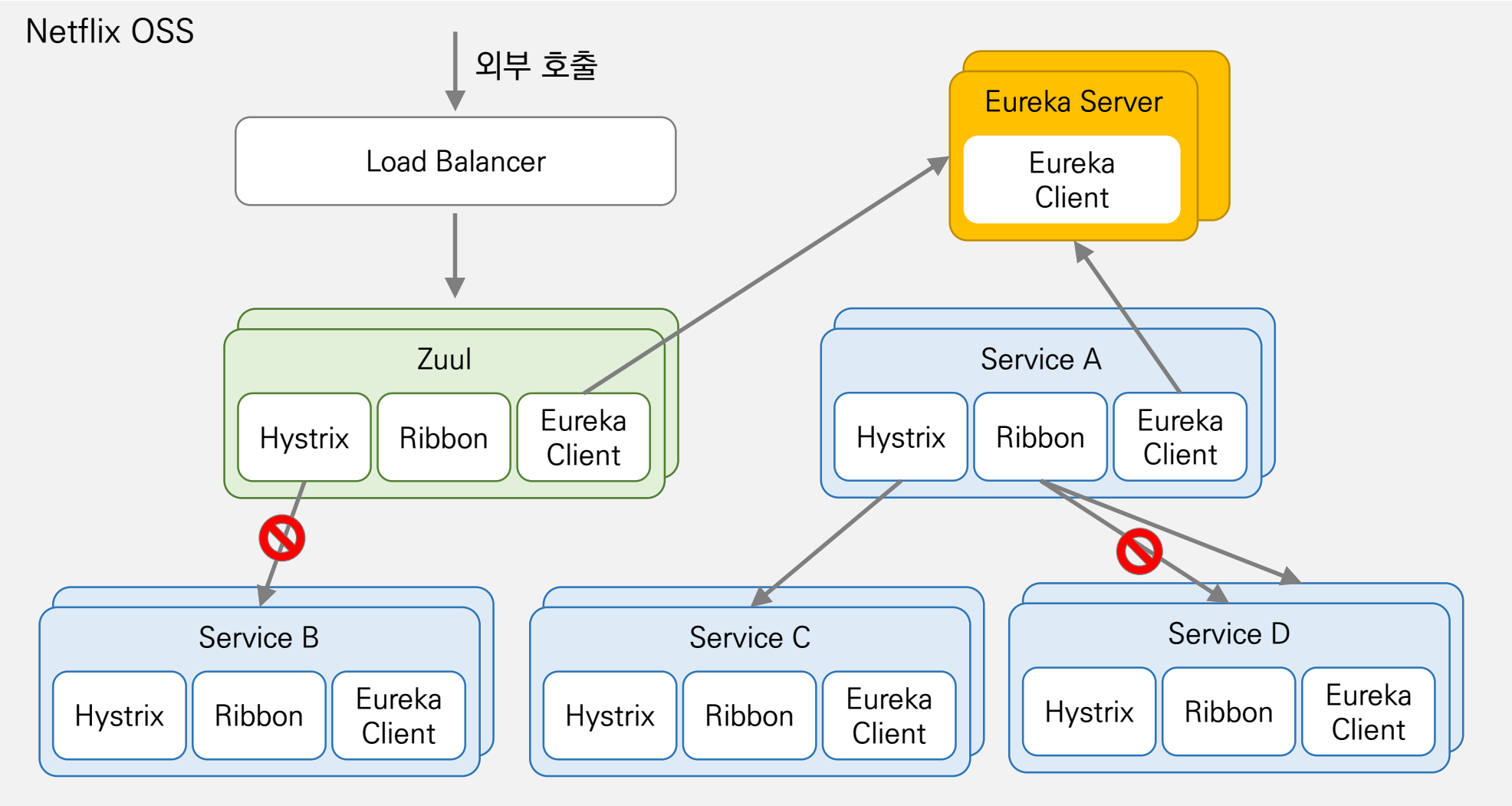
» Netflix OSS 란?

- Netflix OSS는 Netflix Open Source Software의 약자로서 Netflix사에서 이전에 심각한 서버 장애를 겪은 이후로 신뢰성 있고 수평 확장 가능한 클라우드 시스템으로 이전하기 위한 MSA 전환 기술 노하우와 문제해결 방법을 공유한 오픈 소스

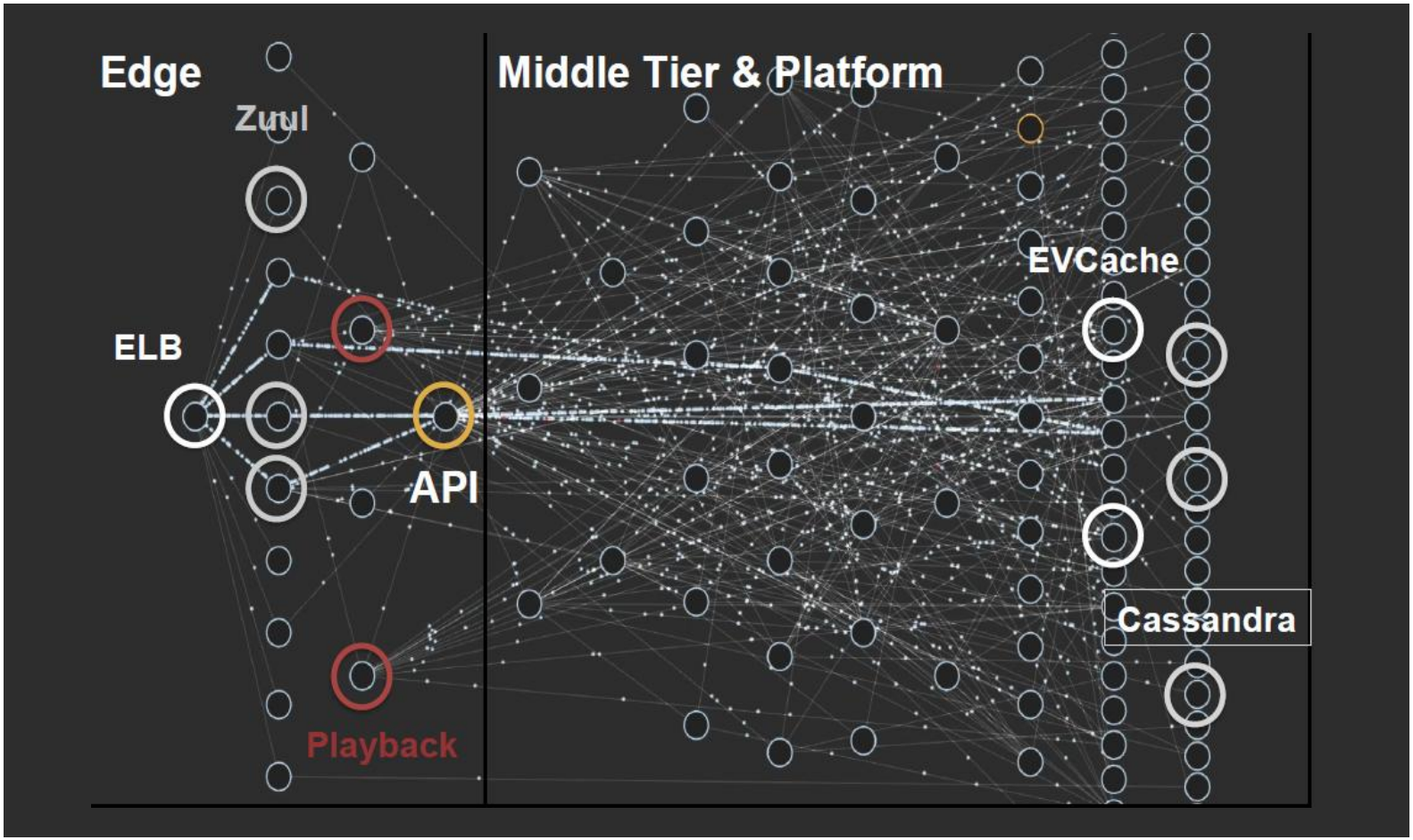
- 대표적인 오픈소스로는 Eureka, Hystrix, Ribbon, Zuul이 있다.

출처 : https://github.com/Netflix

» Netflix MSA Architecture



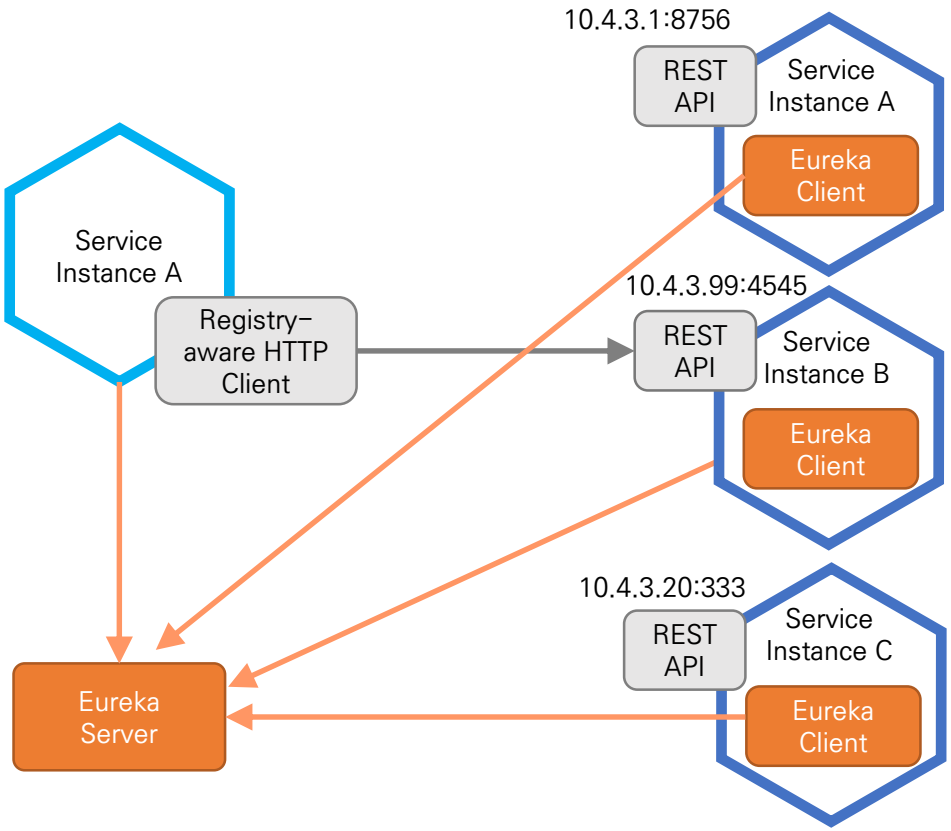
» Netflix MSA 시스템 호출 모습



출처 : https://www.youtube.com/watch?v=CZ3wIuvmHeM

» Eureka

Eureka



- Service Discovery & Registry로서 동적으로 변화하는 서비스 인스턴스를 호출할 수 있도록 서비스 생성 정보를 제공 관리하는 서비스
- Eureka Server와 Eureka Client로 구성
- Eureka Client는 자신의 메타정보(IP, port 등)을 Eureka Server에 등록
- Eureka Server는 각 Client로 부터 30초 마다 Heartbeat를 수신 받아 Health Check를 하여 Client를 관리

» Service Discovery

Service Discovery

- MSA의 경우 인스턴스가 클라우드 환경에서 오토 스케일링으로 인해 동적으로 생성되거나, 컨테이너 기반의 배포로 인해서 서비스의 접속정보가 동적으로 변경
- 서비스의 접속정보를 알아낼 수 있는 기능이 필요하며, 이를 가능하게 하는 것이 Service Discovery
- Service Discovery 방식은 Client Side Discovery와 Server Side Discovery이 있음

Client Side Discovery

- 각 서비스에 Client가 Service Registry(Server)에 접속정보를 등록하는 방식
- 대표적으로 Eureka가 있음
- Client가 사용 가능한 서비스를 알기에 개별로 로드밸런싱 방법을 선택
- Client와 Service Registry(Server)와 의존성 발생

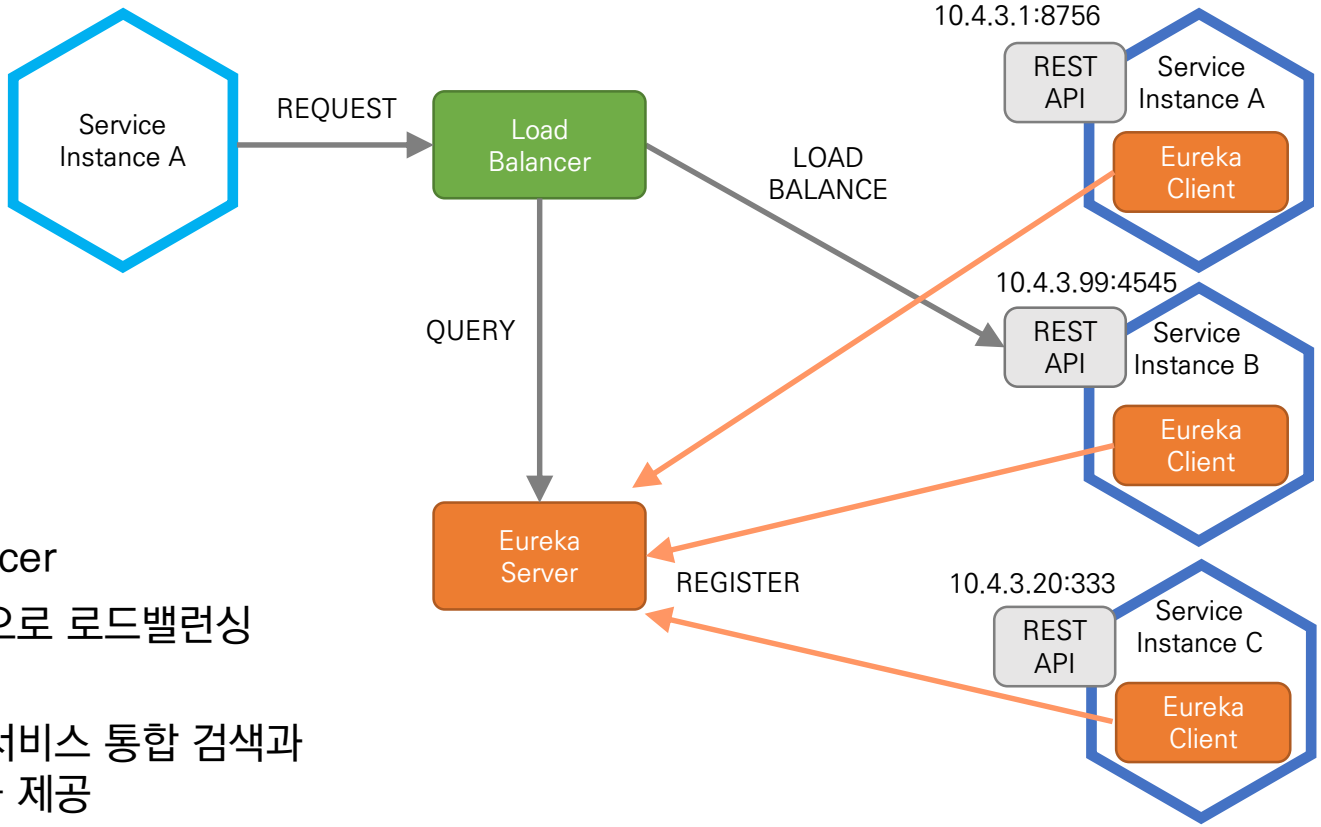
VS

Server Side Discovery

- 호출되는 서비스 앞에 로드밸런서를 두는 방식으로, 이 로드밸런서로 서비스의 위치를 라우팅하는 방식
- 대표적으로 AWS ELB가 있음
- 각 서비스에 Service Discovery를 위한 Client를 제거할 수 있음
- 로드밸런싱을 위한 컴포넌트가 배포되어 있어야 함

» Ribbon

Ribbon



- Client Side Load Balancer
- 여러 서버를 다양한 방식으로 로드밸런싱 기능을 제공
- 로드밸런싱 기능 외에도 서비스 통합 검색과 내결함성을 위한 기능들을 제공

02 Netflix OSS

» Ribbon 구성 요소 및 기능

Server List

- 로드밸런싱 대상 서버 목록
- 설정을 통해 Static하게 설정 가능
- Eureka 등을 기반으로 Dynamic 하게 설정 가능

Rule

- 요청을 보낼 서버를 선택할지 결정하는 Rule을 제공
- Round Robbin : 한 서버 씩 돌아가면서 전달
- Available Filtering : 에러가 많은 서버 제외
- Weighted Response Time : 서버 별 응답 시간에 따라 확률 조절

Ping

- 서버 목록에 있는 서버가 살아 있는지 체크하는 기능
- 서버 목록이 Static, Dynamic 모두 가능

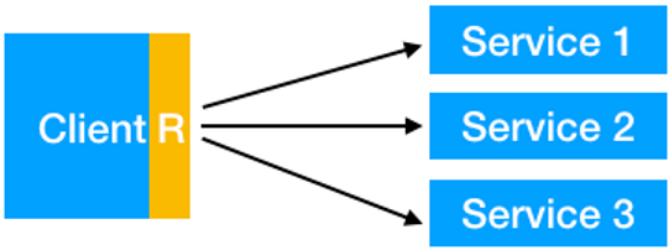
Retry

- 서버에게 응답을 받지 못하였을 경우 동일한 서버로 재시도 하거나 다른 서버로 재시도 하는 기능

» Client Side Load Balancing vs Server Side Load Balancing

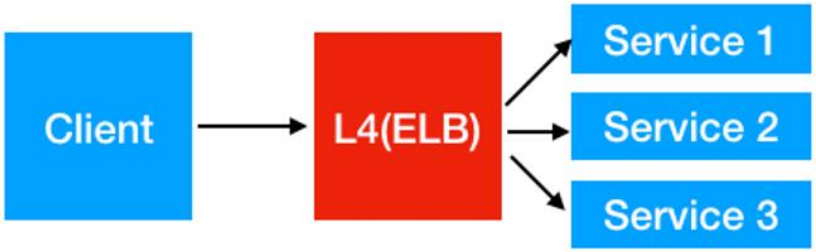
Client Side Load Balancing	VS	Server Side Load Balancing

클라이언트 사이드 로드 밸런싱(Ribbon)



- 물리적인 하드웨어 장비를 이용하는 것이 아니라 클라이언트 프로그램을 통해 로드밸런싱 하는 방식
- 유연하게 확장이 가능
- 서버 목록을 자동으로 추가 및 삭제가 가능

서버 사이드 로드 밸런싱(기존)



- 물리적인 하드웨어(L4 Switch)로 로드밸런싱 하는 방식
- 물리적인 장비를 사용하기 때문에 비용이 많이 소모 되며 유연성 감소
- 서버 목록을 수동으로 추가

출처 : https://sabarada.tistory.com

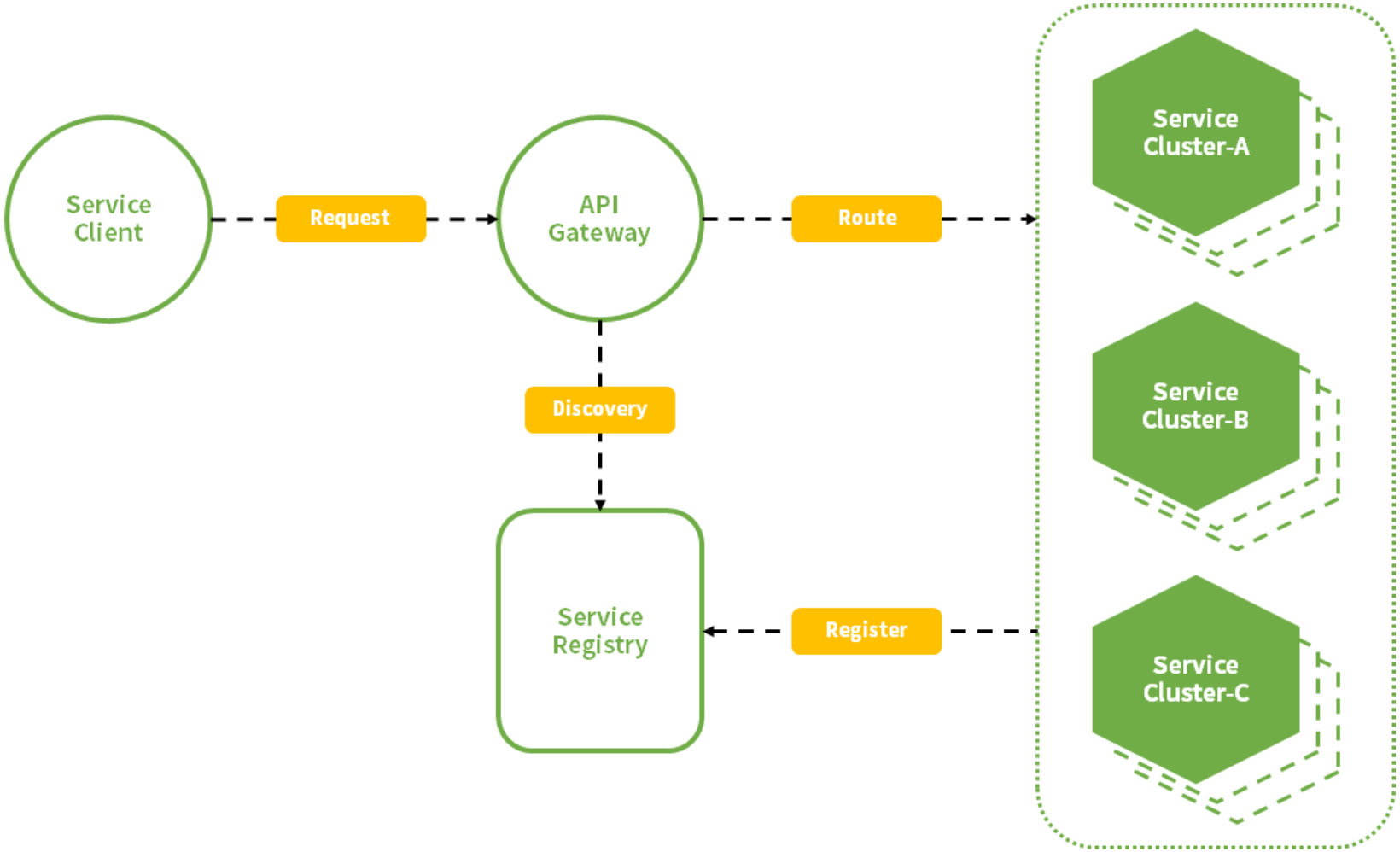
» Zuul

Zuul

- Netflix OSS에서 제공하는 API Gateway
- Netflix OSS는 Zuul을 통해 API 호출 시 단일 엔드포인트 제공
- Zuul은 동적 라우팅, 모니터링, 탄력성, 보안 등을 위한 기능을 제공하는 L7 애플리케이션 게이트웨이
- 단순한 라우팅 이외에도 요청과 응답을 동적으로 가로채 HTTP 메시지를 다룰 수 있는 필터링 기능을 함께 제공

출처 : https://github.com/Netflix/zuul

» Zuul Architecture



출처 : https://yesesyo.medium.com/

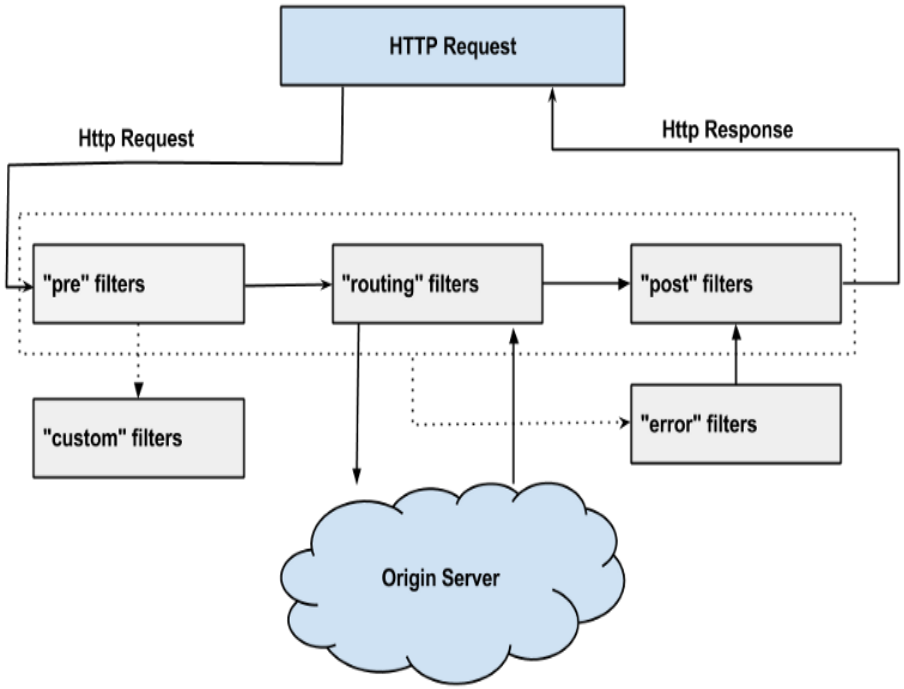
» Zuul 기능

Zuul 기능

- Authentication and Security : 클라이언트 요청 시, 각 리소스에 대한 인증 요구 사항을 식별하고 이를 만족시키지 않는 요청은 거부
- Insights and Monitoring : 의미 있는 데이터 및 통계 제공
- Dynamic Routing : 필요에 따라 요청을 다른 클러스터로 동적으로 라우팅
- Stress Testing : 성능 측정을 위해 점차적으로 클러스터 트래픽을 증가
- Load Shedding : 각 유형의 요청에 대해 용량을 할당하고, 초과하는 요청은 제한
- Static Response Handling : 특정 요청에 대해서는 백엔드로 트래픽을 보내는 대신 즉시 응답하도록 구성 가능

» Zuul Filter

Filter 종류



- **Pre Filter**
  라우팅 이전에 실행되며, 클라이언트 인증/인가 등에 활용
- **Route Filter**
  라우팅을 위해 단순히 URI 패턴에 의존하는 것이 아닌, 보다 복잡한 매커니즘을 이용하고자 할 때 활용
- **Post Filter**
  라우팅 이후에 실행되며, 상태코드와 같이 응답 메시지를 빈번하게 조작해야 할 경우 활용
- **Error Filter**
  필터에서 에러가 발생할 경우 실행

출처 : https://medium.com/netflix-techblog

» REST API 정의

REST API

{ REST API }

Representational State Transfer API

- Representational State Transfer 약자로 자원을 이름으로 구분하여 해당 자원의 상태를 주고받는 모든 것을 의미
- REST API 준수하여 설계된 API를 RESTful API라고 지칭

출처 : https://meetup.toast.com/posts/92

» REST API의 특징

REST API의 특징

- 유니폼 인터페이스(Uniform)
  - HTTP 표준만 따르면 특정 플랫폼(iOS, WEB, 안드로이드), 언어, 기술에 종속되지 않고 사용 가능한 URI로 리소스 조작 가능
- Client – Server 구조
  - REST 서버가 API 제공하여 클라이언트 인증과 같은 기능을 직접 관리하여 클라이언트와 서버의 내용이 명확해 져서 서로간 의존성이 떨어짐
- 무상태성(Stateless)
  - 상태 정보를 따로 관리하지 않아 서비스의 자유도가 높아지고 구현이 간단해짐
- 캐시 가능(Cacheable)
  - HTTP 웹 표준을 사용하기 때문에 기존 인프라를 활용 가능하여 HTTP가 가진 캐싱 기능을 적용 가능
- 자체 표현 구조(Self-Descriptiveness)
  - REST API 메시지만 보고도 이해하기 쉬울 정도로 자체 표현 구조로 되어 있음
- 계층형 구조(Layered System)
  - REST 서버를 다중 계층으로 구성하여 보안, 로드밸런싱, 암호화 계층 등을 추가할 수 있음

» REST API Method

REST API Method

- Application을 API를 통해 통신하는 작은 서비스의 집합으로 개발하는 아키텍처형 접근법

“REST”

GET	/user
GET	/user/:userID
POST	/user
PUT	/user/:userID
DELETE	/user/:userID

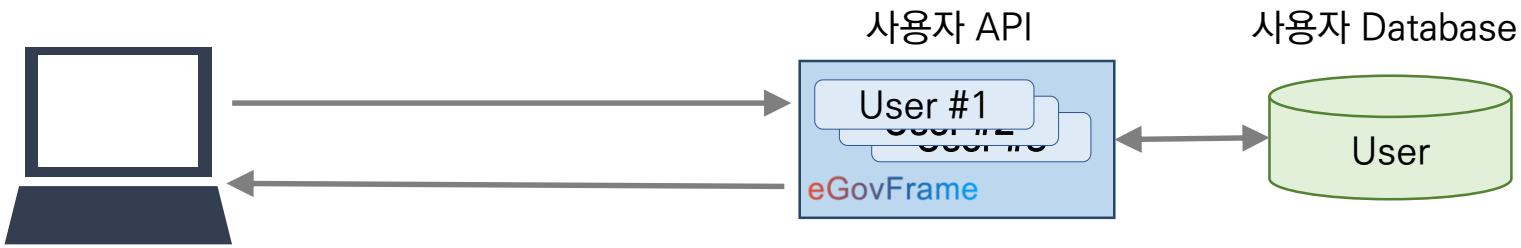
메소드	기능
GET	특정 데이터 혹은 리스트 조회
POST	신규 데이터 생성
PUT	기존 데이터 수정
DELETE	기존 데이터 삭제

» REST API 호출 예시

REST API 호출 예시

- 제어 할 리소스와 작업은 URI와 HTTP 메소드를 이용하여 명시하고, 해당 작업을 위한 Request Parameter와 Response는 Body 영역에 JSON 메시지를 통해 데이터를 주고 받는다.

Request PUT /user/admin
{“userId”:”admin”, “username”:”어드민”}



Response
{“status”:”SUCCESS”, “result”:{“userID”:”admin”, “username”:”어드민”}}

» 우리 시스템에 MSA 도입이 적합한가?

MSA 도입 시 체크 리스트
✔ 마이크로서비스를 요구할 만큼 시스템 복잡도가 높은가?
✔ 여러가지 기술을 적용할 생각이 있는가?
✔ 비즈니스의 변화가 많은가?
✔ 특정 기능 혹은 특정 시간에 트래픽이 몰리는가?
✔ DevOps 문화, 기술을 받아드릴 조직 문화가 구성되어 있는가?
✔ 데이터 관리 방법에 대한 대안을 생각해 두었는가?
✔ 도입할 경우 비용을 얼마나 절감할 수 있는가?
✔ 개발과 운영을 동시에 요구할 만큼 인프라가 준비되어 있는가?

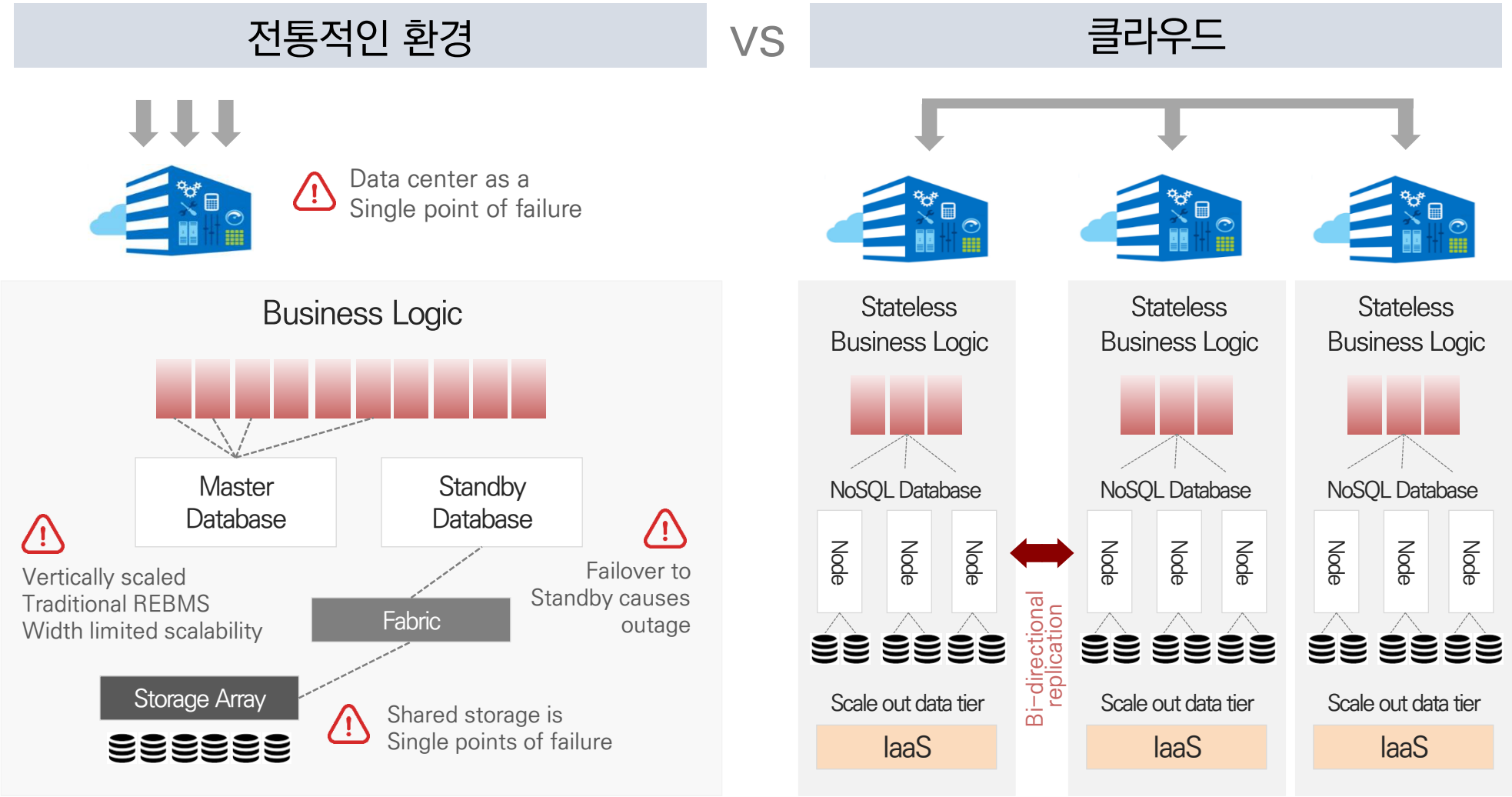
» 기존 시스템과의 병행

새로운 MSA 환경 구성	VS	기존 레거시 환경에 적용
MSA의 장점을 최대한 살릴 수 있도록 인터페이스 환경과 빌드, 배포 환경을 새롭게 구성 도입효과 ▲ 변경영향도 ▲		기존에 구성되어있는 인터페이스 환경과 빌드, 배포 프로세스를 준수하여 구성 도입효과 ▼ 변경영향도 ▼

일부 업무 우선적용 후 단계적 확대	VS	전체업무에 빅뱅방식 적용
신규 개발업무, 독립적 업무 등 적용이 용이한 업무를 우선적용 후 단계적으로 분리		일체형 S/W → 미니서비스 → 마이크로서비스로 전체업무를 단계적으로 분할

Micro Service Architecture의 도입

» MSA 운영 환경에 대한 고민

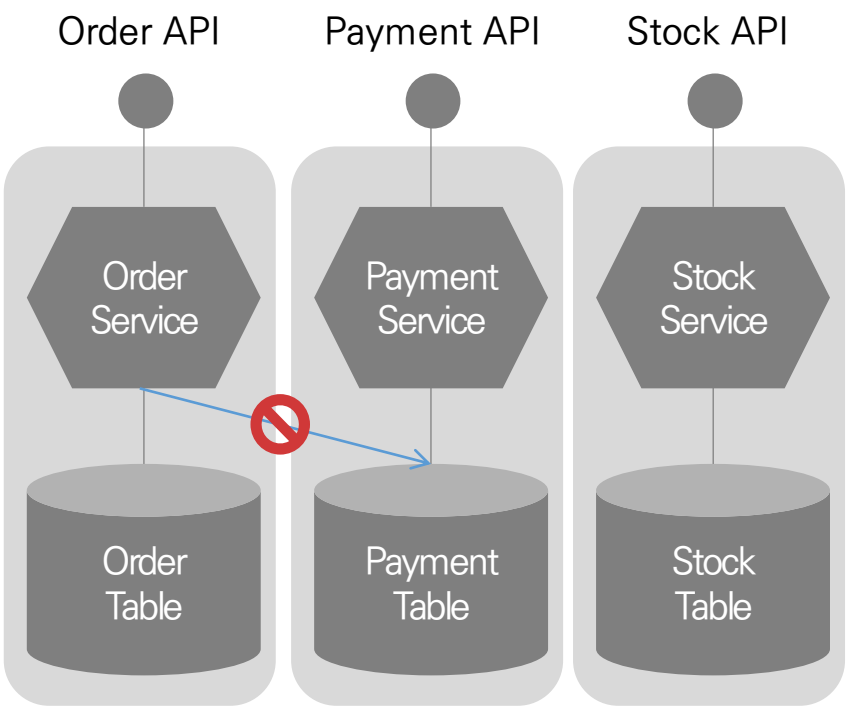


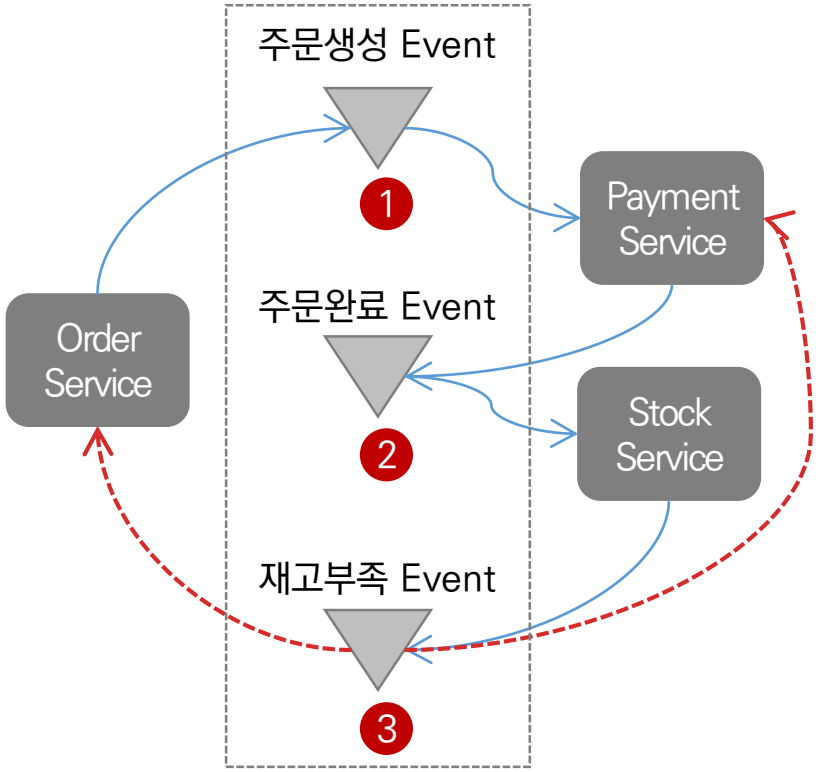
Micro Service Architecture의 도입

» DB 트랜잭션 처리에 대한 고민

Microservice 간 데이터 일관성 처리

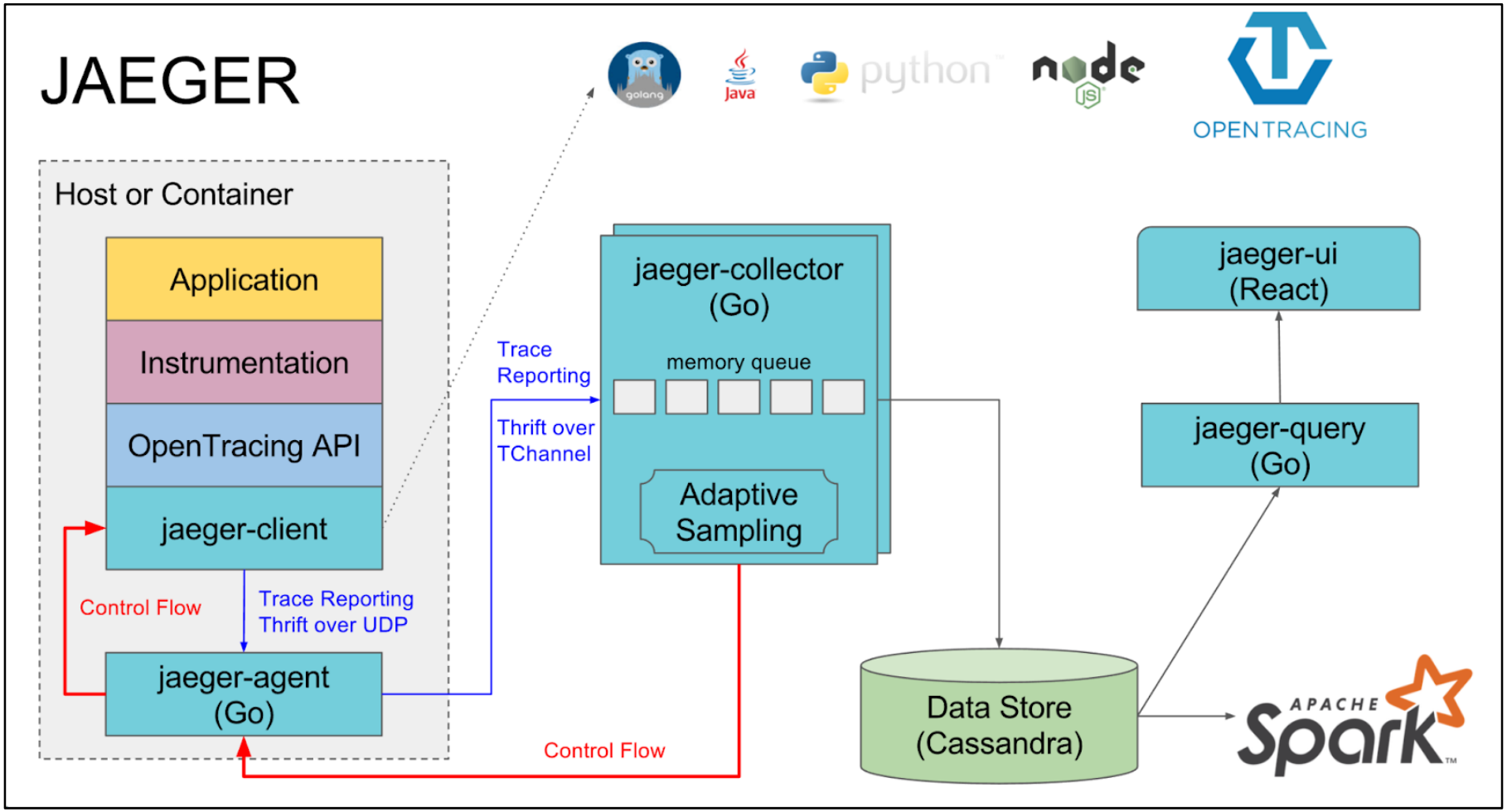
- Microservice별 트랜잭션 처리
- API를 통한 타 서비스 데이터 액세스
- Event-Driven Architecture
- 트랜잭션 실패 시 보상 트랜잭션 처리





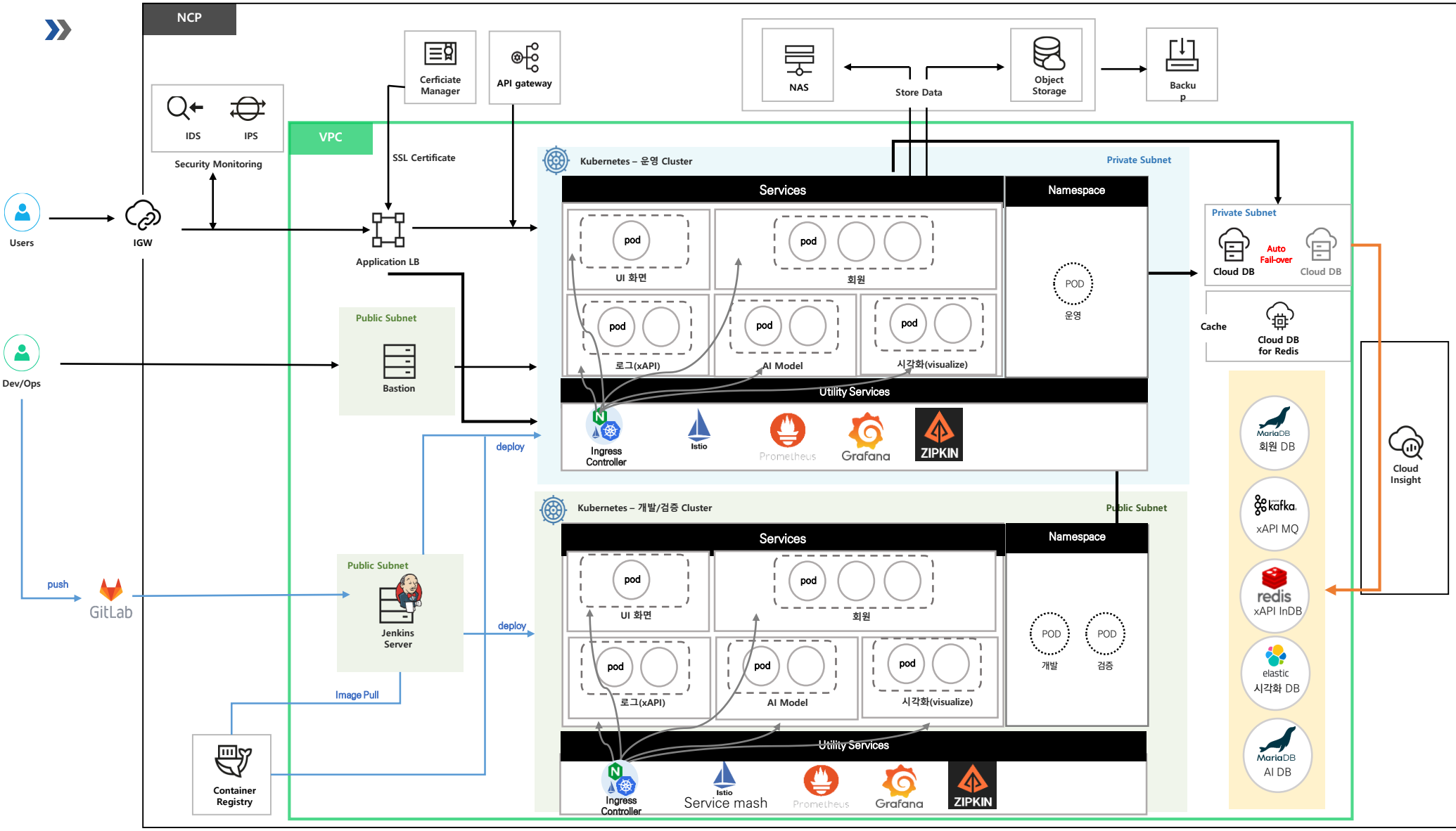
Micro Service Architecture의 도입

» 분산된 로그의 수집 정책 수립

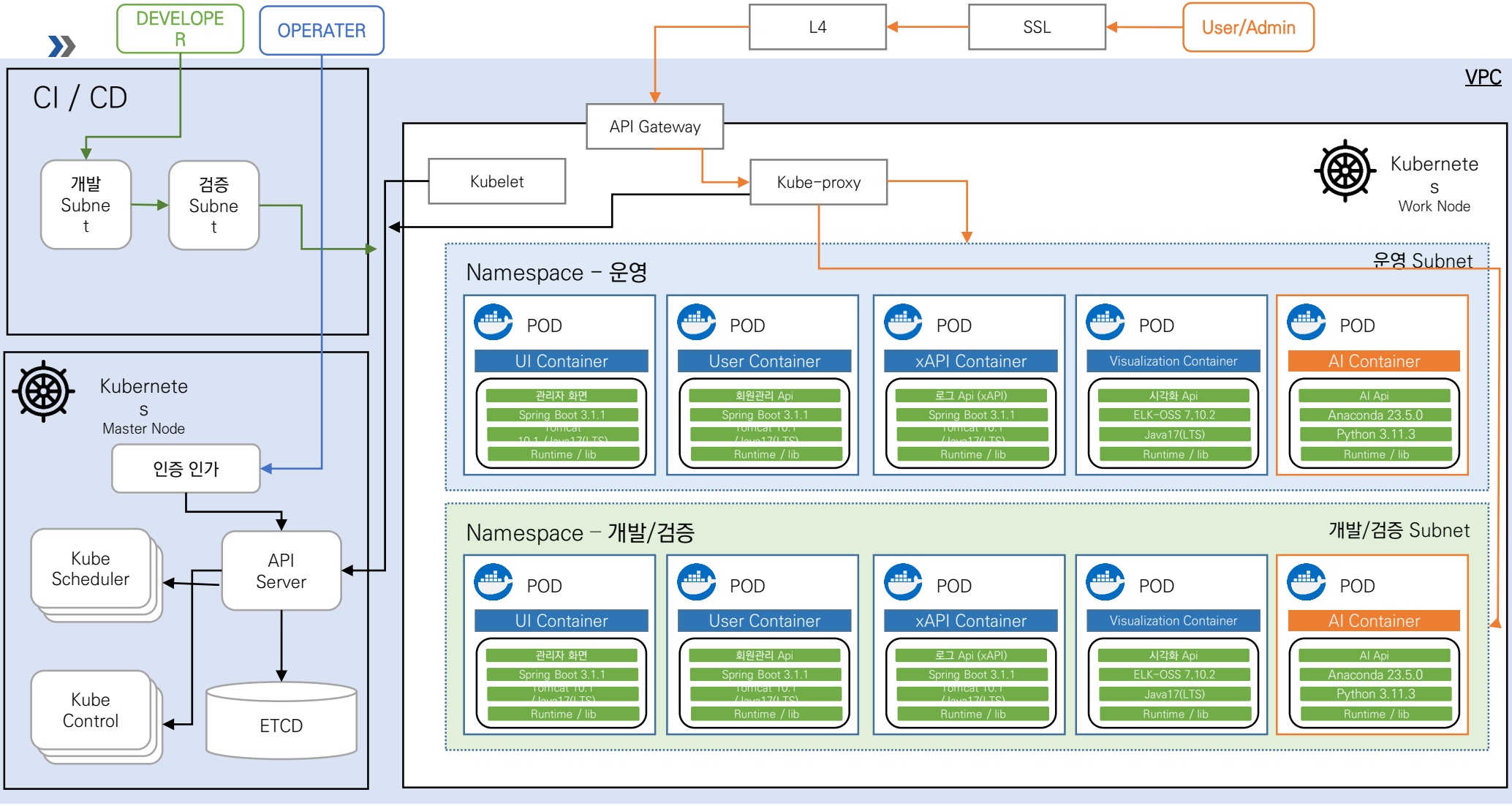


04. MSA 구성 사례

구성 사례 1

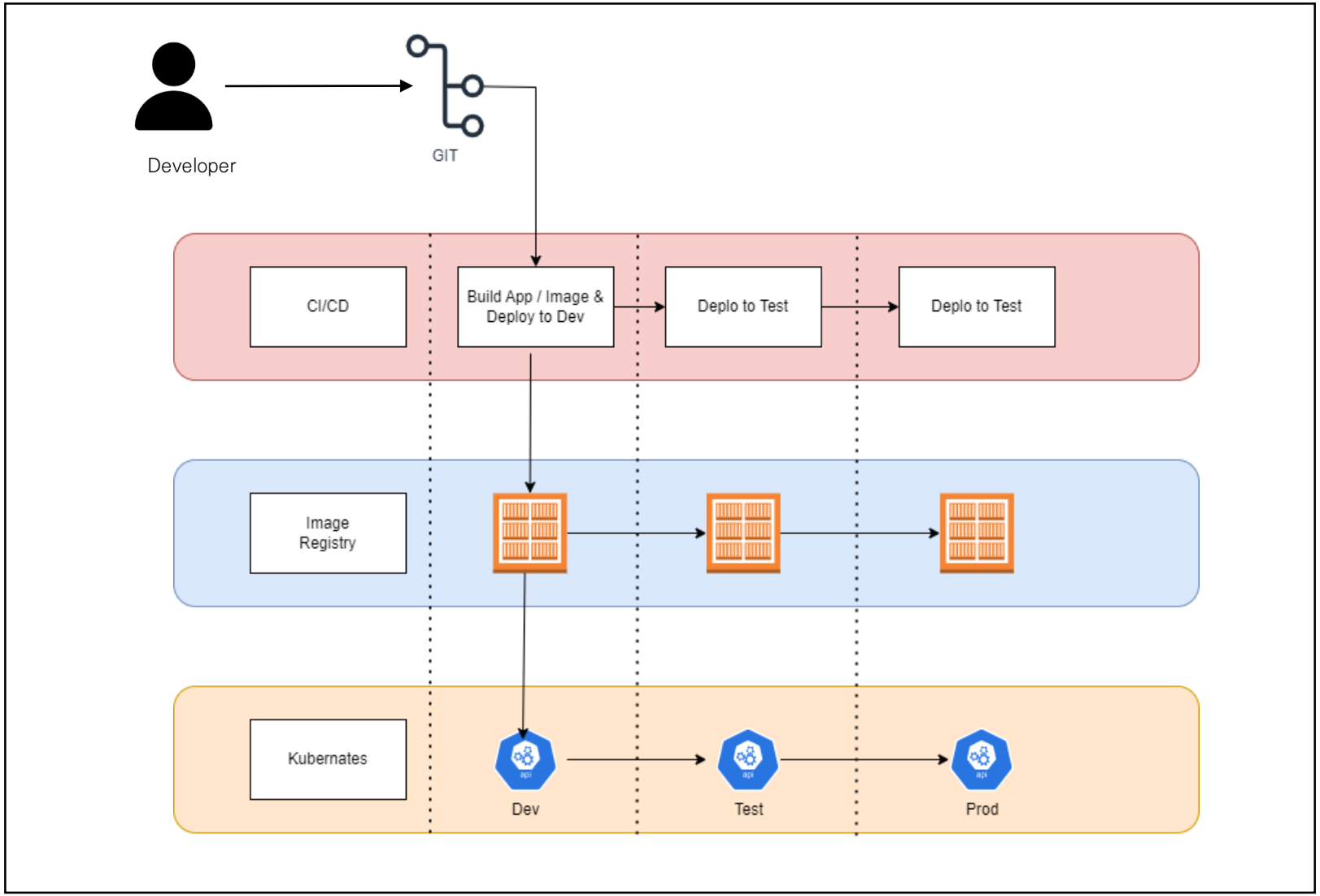

NCP
Cerficiate Manager
API gateway
NAS
Store Data
Object Storage
Backu p
IDS
IPS
Security Monitoring
VPC
SSL Certificate
Users
IGW
Application LB
Kubernetes – 운영 Cluster
Private Subnet
Services
Namespace
pod
UI 화면
회원
POD
운영
로그(xAPI)
AI Model
시각화(visualize)
Utility Services
Ingress Controller
Istio
Prometheus
Grafana
ZIPKIN
Private Subnet
Cloud DB
Auto Fail-over
Cloud DB
Cache
Cloud DB for Redis
Public Subnet
Bastion
Dev/Ops
deploy
MariaDB
회원 DB
kafka
xAPI MQ
redis
xAPI InDB
elastic
시각화 DB
MariaDB
AI DB
Cloud Insight
push
GitLab
Public Subnet
Jenkins Server
deploy
Kubernetes – 개발/검증 Cluster
Public Subnet
개발
검증
Image Pull
Container Registry
Service mash

05 구성 사례 2


DEVELOPER
OPERATER
L4
SSL
User/Admin
VPC
CI / CD
개발 Subnet
검증 Subnet
API Gateway
Kubelet
Kube-proxy
Kubernetes Work Node
Kubernetes Master Node
인증 인가
Kube Scheduler
API Server
Kube Control
ETCD
Namespace – 운영
운영 Subnet
POD
UI Container
관리자 화면
Spring Boot 3.1.1
Runtime / lib
User Container
회원관리 Api
xAPI Container
로그 Api (xAPI)
Visualization Container
시각화 Api
ELK-OSS 7.10.2
Java17(LTS)
AI Container
AI Api
Anaconda 23.5.0
Python 3.11.3
Namespace – 개발/검증
개발/검증 Subnet

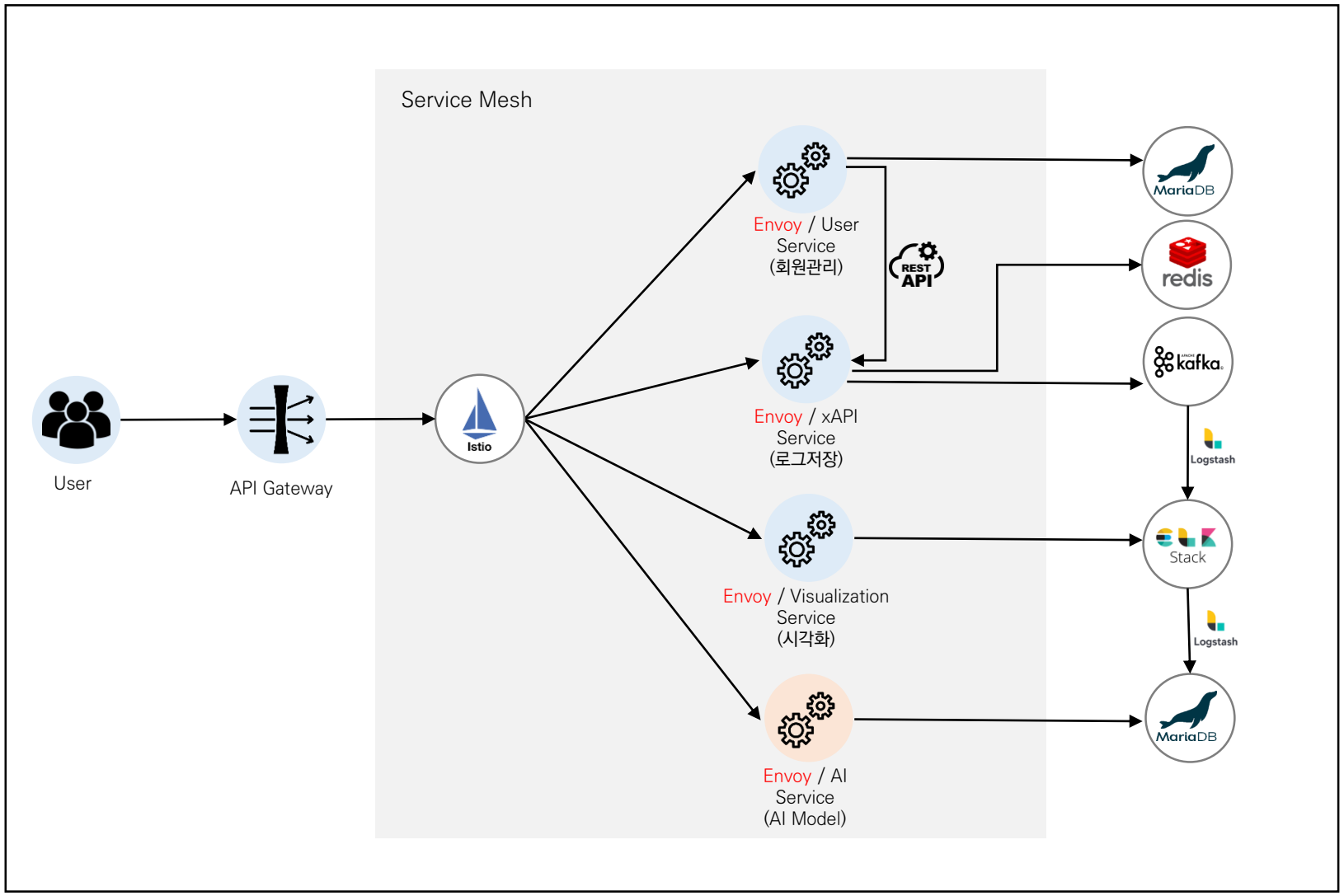
구성 사례 3

» CI / CD Environment



구성 사례 1

서비스 간 호출 관계



감사합니다

THANK YOU